微网与外部电网电力调控策略

摘要：本文以并网微网日内电力调控为研究对象，结合光伏出力、小区负荷、分时电价与多时刻光伏预报数据，综合考虑储能容量、充放电功率、效率约束，以及紧急购电、计划调整违约惩罚、实时波动电价等市场规则，建立递进式混合整数线性规划优化模型体系，并采用 HiGHS 分支定界算法求解。**针对问题一**，建立确定性日循环储能调度模型（模型 M1），以全天购电费用最小为目标，引入 0-1 二元变量 u_t 实现充放电互斥，建立 SOC 动态递推关系与日循环边界约束。采用“LP 求下界 +MILP 求最优”两阶段求解策略。典型日求解结果表明，全天最优购电费用为 35126.9486 元，显著低于无储能基准的 48052.0466 元，储能通过移峰填谷的储电用电策略降低购电成本约 26.9%。**针对问题二**，建立日前计划与实时补救两阶段随机规划模型（模型 M2），采用情景树描述光伏与负荷不确定性，施加非前视约束保证决策因果一致性，以高价紧急购电（5 倍电价）弥补实时缺口，目标为期望总费用最小。求得的最优解中的四个代表时间**针对问题三**，构建多信息集滚动优化模型（模型 M3），在 0:00、6:00、12:00、18:00 四个合法时点调整未执行时段的购电计划，建立上调加价（1.5 倍）与下调违约（50%）的非对称费用核算公式。通过四组嵌套信息集配对统计检验量化滚动预报的信息价值，结果表明完整四批次预报较仅 0 点预报可降低总运行费用约 4.7%，新增预报的经济价值具有统计显著性，有充分理据证明应该使用多批次预报作为决策以替代仅参考 0 点预报的模式。**针对问题四**，将固定分时电价扩展为实时波动电价，分别重构问题二与问题三的优化模型（模型 M4a、M4b），在价格与负荷联合情景树下重新求解。结果显示，电价波动使总购电成本上升约 5.2%，滚动调整机制可抵消约 1.8 个百分点的波动影响。本文通过约束残差审计、多情景交叉验证与灵敏度分析完成模型稳定性与鲁棒性检验。结果表明，建构的模型能够有效兼顾供电可靠性与运行经济性，可为微网电力调控的实际场景提供参考依据。

关键词：微网；光伏发电；储能调度；混合整数线性规划；滚动优化

1 问题重述

1.1 问题背景

在“双碳”目标推进和新型电力系统建设背景下，分布式光伏在居民小区、产业园区等场景中加速普及。并网型微网能够整合分布式电源、自身储能与外网交互，是降低电网波动和提升供电可靠性的重要方式。然而，光伏出力受气象条件影响，具有明显的间歇性、波动性、季节性；小区负荷又受居民作息和季节气候影响，呈现早晚高峰、午间低谷等特征。二者在时序上出现了错配：正午光伏出力较大但负荷偏低，容易出现本地电力富余；傍晚至夜间失去光伏，负荷进入高峰，微网不得不增加外网购电。这样的方式一方面可能由于购买过量造成弃光，另一方面也会推高高电价时段的购电成本。

储能设备是缓解源荷错配的关键。通过在低价或光伏富余时段充电，在高峰或高电价时段放电，可降低整体购电费用。但储能运行受额定容量、最大充放电功率以及安全电量区间等条件限制，不能无约束充放电。同时，微网还面临分时电价、紧急购电、滚动更新的光伏预报、购电计划调整带来的费用以及外网电价波动等市场规则约束。物理约束与市场机制共同制约，使调度难以兼顾供电需求与经济性。

基于上述背景，本文围绕微网与外部电网的电力调控问题，依次开展确定性调度、不确定场景下日前计划、滚动预报调整与波动电价拓展四项研究。具体问题如下：

问题 1：在每日电价与小区负荷固定、光伏发电功率预测已知的条件下，于当日 0:00 制定全天购电计划。要求微网供电量不低于小区负荷，储能设备在 0:00 与 24:00 储电量相等，在满足储能运行约束的前提下最小化全天购电费用，并给出具体的购电与储能充放电方案。

问题 2：在电价固定、小区负荷与光伏发电功率逐时变化的场景下，每日 0:00 制定当天计划购电策略。若计划供电量低于小区负载，需向外网紧急购电，紧急购电电价为该时刻交易电价的 5 倍；其余时段购电费用按计划购电量计算。需给出指定日期的计划购电量、储能充放电及紧急购电结果。

问题 3：在每日 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点光伏发电功率预报的条件下，先根据 0:00 预报制定当日初始购电计划，再根据后续时刻的更新预报调整购电策略。若调整购电量低于原计划，下调部分按交易时刻电价的 50% 收取违约费用；若调整购电量高于原计划，超出部分按交易时刻电价的 1.5 倍计费。总购电费用包括计划购电费用、紧急购电费用和调整购电量相关费用。需建立相应数学模型，并分析引入多时刻预报的经济价值。

问题 4：考虑外网电价实时波动的情形，建立微网当日购电策略的数学模型。基于小区负荷、光伏发电、滚动光伏预报与实时电价数据，分别重新求解问题 2 与问题 3，给出波动电价下的购电与储能调控结果。

1.2 任务拆解

本文按照“确定性基准—不确定性补救—滚动优化扩展—波动电价拓展”的递进逻辑，将四个问题拆解为以下核心任务：

- 问题 1：以典型日固定负荷、固定电价与光伏预测为输入，决策日内各时段购电量与储能充放电量，在满足供电平衡、储能物理约束与日循环边界条件下，最小化全天购电总成本，形成确定性调度基准。
- 问题 2：考虑负荷与光伏的日内不确定性，在每日 0:00 确定全天固定购电计划，日内通过高价紧急购电弥补实时供电缺口，决策目标为期望总费用最小，同时保证决策不利用未来真实信息。
- 问题 3：引入多时刻滚动光伏预报，在 0:00、6:00、12:00、18:00 四个合法时点对未执行时段的购电计划进行调整，考虑上调加价与下调违约的非对称费用，量化新增预报信息的经济价值。
- 问题 4：将固定分时电价扩展为实时波动电价，分别重构问题 2 与问题 3 的优化模型，在电价不确定场景下重新求解调度策略，对比电价波动对购电成本、储能运行与调整行为的影响。

2 模型假设与符号说明

2.1 模型假设

为简化建模过程、保证模型物理可实现性与经济合理性，结合题目条件与工程实际，作出如下假设与建模前提：(1) 题目附件所提供的原始数据为建模唯一输入依据；经时序完整性与异常值校验，数据时序连续、无缺失记录与明显异常值，建模过程直接采用原始数据，不做插值补全与人为修正。(2) 研究周期内无不可预见的重大外部突发事件干扰系统运行，不考虑电网故障、极端天气等极端场景。(3) 储能设备充、放电单程效率均为 90%，调度周期内无自放电、无容量衰减，不考虑充放电次数限制。(4) 微网仅可从外部电网购电，不具备向电网售电的能力；本地富余光伏可免费弃置，不产生额外收益或惩罚。(5) 光伏预报与购电计划调整严格遵循因果性：决策仅可使用当前已发生的观测与已发布的预报，不可利用未来真实数据。(6) 紧急购电仅用于弥补实时供电缺口，不得用于储能充电；同一时段内储能充电与放电互斥，不可同时进行。(7) 时间数据采用区间端点标记，00:10 代表刚结束的十分钟段，末列对应次日 00:00；所有调度与结算按十分钟区间聚合执行。(8) 问题 1 储能初末状态固定为 6000 kW · h 并满足日循环；其余问题储能状态跨日连续传递，不强制每日重置。

2.2 符号说明

本文主要符号定义如表 1 所示。

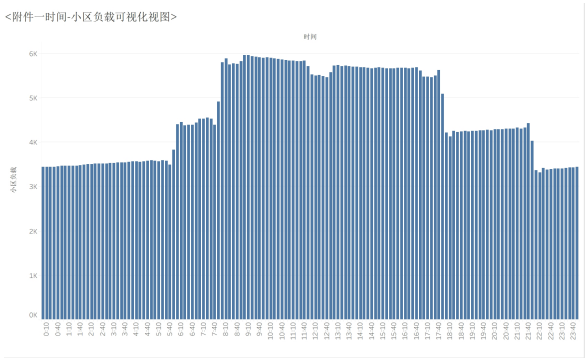
表 1: 主要符号说明

符号	含义	单位
d, t	日期索引、日内时段索引	—
k	修订时刻/决策时刻索引	—
Δ	时间步长，对应 10 分钟	h
$\mathcal{T}$	允许购电计划修订的时刻集合	—
$\mathcal{D}_{\text{out}}$	输出统计日期集合	—
L_t	t 时段小区负荷电量	kW · h
V_t	t 时段光伏发电电量	kW · h
p_t^0	t 时段固定分时电价	元/kW · h
p_t	t 时段实际交付电价	元/kW · h
$p_t(\omega)$	情景 ω 下 t 时段实际交付电价	元/kW · h
B_t	t 时段初始计划购电量	kW · h
$Q_t^{(k)}$	第 k 次修订后的 t 时段购电量	kW · h
Q_t^*	t 时段调整后最终购电量	kW · h
c_t	t 时段储能充电量	kW · h
r_t	t 时段储能放电量	kW · h
e_t	t 时段紧急购电量	kW · h
W_t	t 时段未利用富余电量	kW · h
E_t	t 时段起始时刻储能储电量	kW · h
$E_{\min}, E_{\max}$	储能运行电量下限、上限	kW · h
$P_{\max}$	储能最大充放电功率	kW
U	单时段最大充放电能量	kW · h
η_c, η_d	储能充电效率、放电效率	—
u_t	储能充电模式 0-1 变量	—
a_t^+, a_t^-	购电计划上调量、下调量	kW · h
ω, π_ω	情景索引、对应情景概率	—
Φ_k	储能终端价值函数	元
C_i	问题 i 全天购电总费用	元
δ_d	单日少/多信息集的费用差	元
$\bar{\delta}$	日均费用差（信息价值均值）	元
Δe_d	单日紧急购电量差值	kW · h
$\mathbb{E}[\cdot]$	数学期望算子	—
$\bar{x}$	时序序列均值	对应原变量单位
ρ_{xy}	皮尔逊相关系数	—

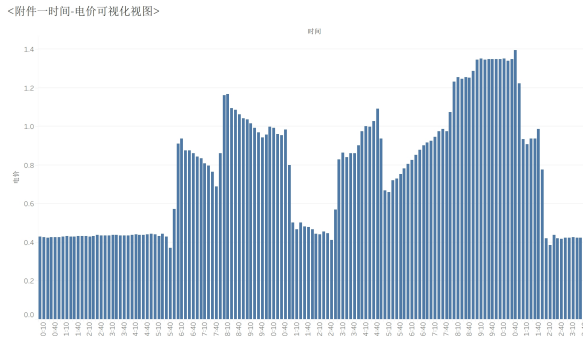
3 数据预处理

3.1 数据质量分析与基础特征

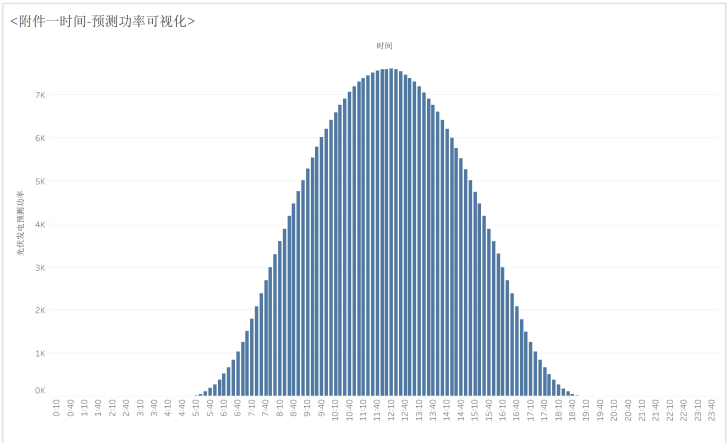
本文对附件提供的负荷、电价、光伏出力与预报数据开展时序可视化与质量校验，明确数据特征与适用边界，为后续建模提供基础。



(a) 小区日内负荷时序曲线



(b) 分时电价日内变化曲线



(c) 光伏发电功率日内预测曲线

图 1: 小区负荷、分时电价与光伏发电功率日内时序特征

对图 1 中各子图进行分析。子图 1a 显示小区用电负荷在早晨 8 点至下午 18 点处于高峰，夜间处于低谷，与居民作息规律一致；子图 1b 反映电价存在明显峰谷差异，早高峰时段电价较高，午间电价处于低位，与用电负荷峰谷特征对应；子图 1c 为日内光伏发电预测曲线，呈现正午出力最大、夜间出力为 0 的典型特征。

为定量描述数据特征，对附件 1 中 144 个时段的负荷、光伏、电价分别计算均值、极值与相关系数。对于长度为 N 的时序序列 $\{x_t\}$，其均值计算公式为：

$$\bar{x} = \frac{1}{N} \sum_{t=0}^{N-1} x_t \tag{1}$$

经计算，附件 1 典型日数据的统计特征如下：

- 小区负荷功率范围为 3309.3934—5958.9696 kW，均值为 4626.0337 kW；

- 光伏发电功率范围为 0—7612.3160 kW，均值为 2311.7848 kW；
- 分时电价范围为 0.3713—1.3952 元/kWh，均值为 0.7662 元/kWh。

计算各序列间的皮尔逊相关系数：

$$\rho_{xy} = \frac{\sum_t (x_t - \bar{x})(y_t - \bar{y})}{\sqrt{\sum_t (x_t - \bar{x})^2 \sum_t (y_t - \bar{y})^2}} \tag{2}$$

得到负荷与光伏的相关系数为 0.8564，电价与负荷的相关系数为 0.4425，电价与光伏的相关系数为 0.0836。

对比三条时序曲线可以发现源-荷-电价三者存在时序错配现象：正午光伏出力较大但负荷偏低，本地电力富余；傍晚光伏出力下降而负荷抬升，此时电价也处于高位，因此需要依靠储能实现电能时间平移，降低高峰购电支出。

为保障建模输入的可靠性，本文从时序完整性、数值合理性、物理一致性与单位匹配四个维度对原始数据集开展质量校验：**第一，时序完整性：**数据集时间粒度为 10 分钟，时间轴连续完整，各附件数据均无缺失记录与时间断点；附件 1 包含典型日 144 个时段的负荷、电价与光伏数据，附件 2、附件 4 包含全年逐时段负荷、光伏与电价数据，附件 3 包含多批次滚动光伏预报数据，时间标签对齐一致。**第二，数值合理性：**负荷、光伏出力、购电量、电价均为非负数值，未出现物理意义上的负值；光伏出力在夜间时段均为 0，符合发电特性；电价与负荷的变化范围均处于合理区间。**第三，物理一致性：**负荷、光伏功率单位统一为 kW，时间步长为 10 分钟，电量与功率换算关系一致；储能容量、功率等设备参数与题目给定参数完全匹配。**第四，异常值检验：**采用阈值法与时序连续性检验，未发现明显离群异常点与跳变错误。

综上，各附件数据整体质量可靠，时间维度与物理量纲统一，可直接用于后续微网优化调度建模计算。

4 问题一：确定性日循环储能调度模型

4.1 问题分析

问题一为确定的日内调度优化问题。输入为附件 1 中某日的负荷、分时电价与光伏发电功率曲线，需要在当日 0:00 制定全天每 10 分钟的购电计划与储能充放电计划，在满足供电不低于负荷、储能运行约束的前提下，使全天购电总费用最低，同时要求储能设备在 0:00 与 24:00 的储电量严格相等。

该问题的本质是带物理约束的成本最小化问题。核心难点在于储能电量是随时间动态变化的状态量，必须建立完整的电量递推关系与运行边界；若仅采用普通线性规划建模，缺少对充放电行为的互斥限制，求解结果会出现同一时段电池同时充电、放电的非物理可行解，产生虚假套利。同时日循环边界条件要求调度周期初末储电量相等，与充放电效率、功率上限等约束相互耦合。

因此本文采用混合整数线性规划框架建模，引入 0-1 二元变量实现充放电互斥，同时建立完整的 SOC 动态递推关系与运行边界约束。

4.2 模型建立

问题一中计划购电量即为有效购电量，无紧急购电。模型目标为最小化全天计划购电总费用：

$$\min C_1 = \sum_{t=0}^{143} p_t^0 B_t \tag{3}$$

模型约束条件如下：**（1）微网供需守恒约束**。微网提供的电能不低于小区负荷，用非负剩余量将其写成守恒等式：

$$B_t + V_t + r_t = L_t + c_t + W_t, \qquad B_t, c_t, r_t, W_t \geq 0 \tag{4}$$

（2）储能电量动态递推（SOC 动态模型），刻画储能电量随充放电行为的状态转移：

$$E_{t+1} = E_t + \eta_c c_t - \frac{r_t}{\eta_d} \tag{5}$$

（3）储能运行电量边界。全部 145 个状态端点均受约束，储能额定容量为 12000 kW · h，实际运行上界为 10800 kW · h，不能用额定容量直接替代运行边界：

$$E_{\min} \leq E_t \leq E_{\max}, \qquad t = 0, 1, \ldots, 144 \tag{6}$$

（4）储能充放电功率上限与互斥约束。二元变量 u_t 实现充放电互斥，规避线性规划下同时充放电的非物理解：

$$0 \leq c_t \leq U u_t, \qquad 0 \leq r_t \leq U(1 - u_t), \qquad u_t \in \{0, 1\} \tag{7}$$

其中单时段最大充放电能量 $U = P_{\max} \cdot \Delta = 5000/6$ kW · h。

（5）状态依赖的可充放界。当前可充入与可放出的电量随储能剩余电量实时计算，不能静态化处理：

$$c_t \leq \min\left\{U, \frac{E_{\max} - E_t}{\eta_c}\right\}, \qquad r_t \leq \min\left\{U, \eta_d(E_t - E_{\min})\right\} \tag{8}$$

（6）日循环边界约束。问题一要求储能设备在 0:00 与 24:00 储电量相同：

$$E_0 = E_{144} = 6000 \text{ kWh} \tag{9}$$

4.3 求解策略与正确性保证

本文采用“线性规划求下界 + 混合整数规划求最优”的两阶段求解策略，求解器采用 HiGHS 分支定界算法。首先去掉整数互斥约束求解纯线性规划，得到理论下界，用于后续 MILP 结果的最优性校验；若 LP 解恰好满足充放电互斥，则其本身就是 MILP 的最优解。若 LP 解不满足整数约束，则运行 HiGHS 分支定界求解，主参数 $mip_rel_gap = 1 \times 10^{-7}$，单次求解时间上限 60 秒。

为保证结果正确，本文建立三重校验机制：**基准校验**：最优费用不高于无储能基准费用；**最优性校验**：MILP 目标值不低于 LP 下界，且间隙满足精度要求；**物理约束校验**：供需平衡、SOC 动态、电量边界、充放电互斥、日循环边界全部通过残差审计。

4.4 求解结果与分析

将附件 1 典型日数据输入上述 MILP 模型，采用 HiGHS 求解器完成优化计算，模型求解达到设定间隙精度，全部硬约束满足。完整 144 个时段调度结果保存至 `result1.xlsx`。

按照题目要求，提取指定 6 个时段的购电量结果如表 2 所示。

表 2: 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量（kW · h）	时间段	购电量（kW · h）	时间段	购电量（kW · h）
10:00-10:10	0.000	12:00-12:10	480.4125	14:00-14:10	0.000
16:00-16:10	445.4317	18:00-18:10	531.8941	20:00-20:10	0.000
全天购电量（kW · h）		59482.6990	全天购电费（元）		35126.9486

按 6 个四小时时间块累计的充放电量及端点储电量如表 3 所示。

表 3: 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量（kW · h）	放电量（kW · h）	时间段	充电量（kW · h）	放电量（kW · h）
0:00-4:00	4500.000	0.000	4:00-8:00	833.3333	6365.8412
8:00-12:00	4787.9643	1702.9970	12:00-16:00	5286.0352	91.1014
16:00-20:00	0.000	5780.1319	20:00-24:00	5333.3333	2859.8681
0:00 储电量（kW · h）		6000	24:00 储电量（kW · h）		6000

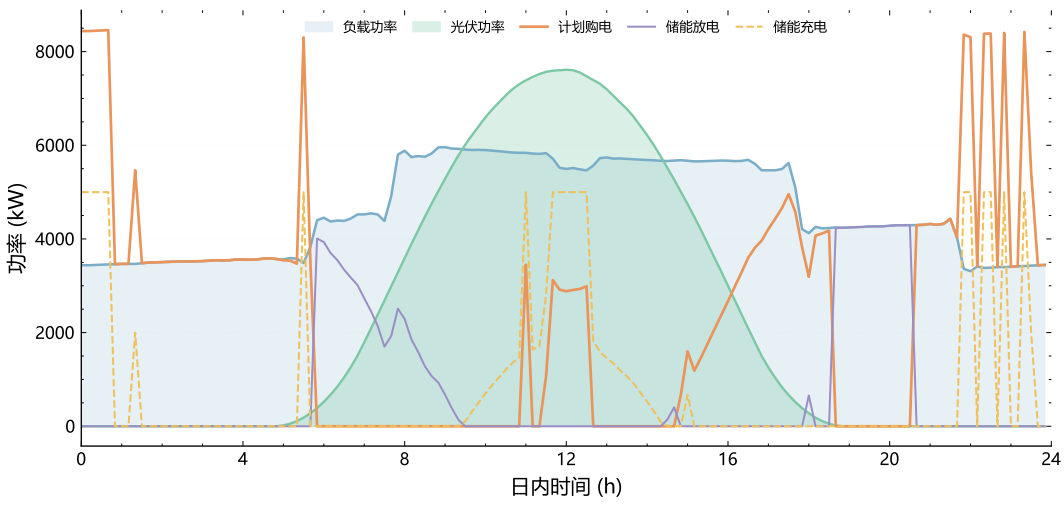


图 2: 问题一典型日内调度时序曲线

由图 2 可见，夜间电价低谷时段微网执行购电并对储能充电；午间光伏发电出力充足，优先就地消纳光伏；傍晚光伏跌落、负荷抬升的用电高峰，储能放电，削减高价时段外网购电量，体现储能移峰填谷的调度逻辑。

图 3 进一步展示了储能电量与电价的联动关系：在电价低谷时段储能充电，电价高峰时段

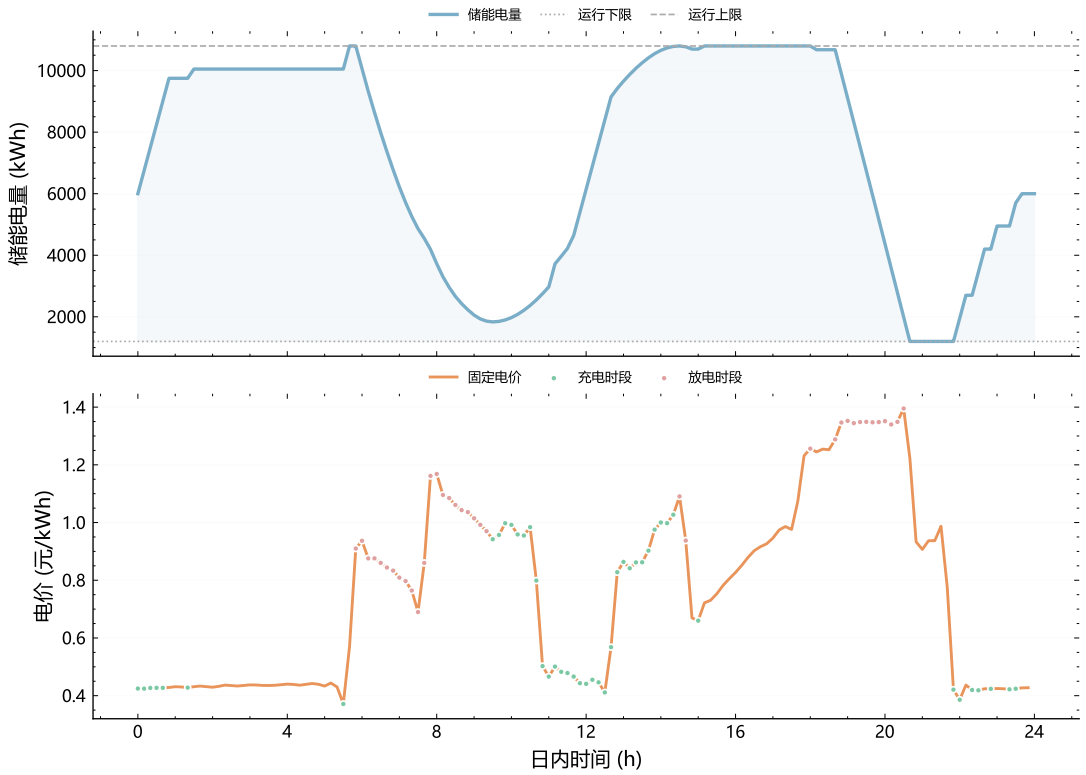


图 3: 储能 SOC 与分时电价日内变化

储能放电，SOC 始终运行在 1200—10800 kWh 安全区间内，验证了“低价充、高价放”的运行策略。

优化后全天购电费用为 35126.9486 元，显著低于无储能基准的 48052.0466 元，降幅达 26.9%。储能严格遵守日循环边界条件，$E_0 = E_{144} = 6000$ kW · h，全程储电量维持在安全运行区间内；约束残差处于机器浮点误差量级，供需守恒、储能递推等式均严格成立，全天无紧急购电。

同时开展约束复核审计：逐条校验充放电互斥条件，不存在同一时段同时充电、放电的非物理解；所有时段购电、充电、放电、弃光量均非负，符合模型物理设定。完整逐时段数据、约束残差与结果校验均随 `result1.xlsx` 同步输出。

5 问题二：日前计划与实时补救模型

5.1 问题分析

问题二为日前计划与实时补救两阶段序贯决策问题。每日 0:00 制定当天购电计划，计划一旦确定全天冻结不可修改；实际运行中负荷与光伏存在不确定性，计划供电不足时通过高价紧急购电弥补缺口，紧急购电价格为正常电价的 5 倍。决策目标为期望总费用最小。该问题的核心难点在于决策必须遵循现实原则，日初制定计划时仅可使用已发生的历史数据，不能利用当日未来时刻的真实负荷与光伏数据；同时计划为全天公共量，不能针对每个情景单独制定。因此本文采用情景树描述未来不确定性，建立两阶段随机规划模型，同时施加非前视约束，保证决策的因果一致性。

为使情景树建立在合法信息集之上，本文采用历史加权滚动预测构造不确定性，采用最近 28 个已完成历史日样本，权重半衰期取 7 日。对变量 X，决策时刻 k 的历史加权预测为：

$$\hat{X}_{d,t|k} = \sum_{j\in\mathcal{D}_k} w_j X_{j,t}, \quad w_j = \frac{2^{-\ell_j/7}}{\sum_{i\in\mathcal{D}_k} 2^{-\ell_i/7}} \tag{10}$$

式中 $\mathcal{D}_k$ 为决策时刻 k 可用的历史日期集合，ℓ_j 为历史日距离当前决策日的天数。在此预测基础上，提取已兑现的负荷、光伏同日联合预测残差，按照六小时块递归分裂生成情景子集，以样本占比作为情景概率，最多保留 16 个叶情景；同一历史节点下的全部情景共享当前时段决策变量，不同分支仅随已揭示的实测信息分化，从情景树构造层面落实非前视约束，确保日初计划与日内补救策略均不会利用未来时刻真实数据。

5.2 模型建立

5.2.1 两阶段随机规划框架

将全天 144 个时段划分为日前决策阶段与实时运行阶段。第一阶段决策变量为全天购电计划 B_t，为一组日前承诺量，不随情景变化；第二阶段决策变量为各情景下的储能充放电量 $c_t(\omega)$、$r_t(\omega)$ 及紧急购电量 $e_t(\omega)$，依赖情景实现值。目标函数为期望总费用最小：

$$\min \sum_{\omega\in\Omega} \pi_\omega \left[\sum_{t=0}^{143} p_t^0 B_t + 5 \sum_{t=0}^{143} p_t^0 e_t(\omega) \right] \tag{11}$$

5.2.2 约束条件

（1）供需守恒约束（逐情景）：

$$B_t + e_t(\omega) + V_t(\omega) + r_t(\omega) = L_t(\omega) + c_t(\omega) + W_t(\omega) \tag{12}$$

（2）SOC 动态递推（逐情景）：

$$E_{t+1}(\omega) = E_t(\omega) + \eta_c c_t(\omega) - \frac{r_t(\omega)}{\eta_d} \tag{13}$$

（3）储能运行边界与充放电互斥：

$$E_{\min} \le E_t(\omega) \le E_{\max}, \quad 0 \le c_t(\omega) \le U u_t(\omega), \quad 0 \le r_t(\omega) \le U(1-u_t(\omega)) \tag{14}$$

（4）紧急购电非负约束：

$$e_t(\omega) \ge 0 \tag{15}$$

（5）非前视约束：第一阶段购电计划 B_t 为全天承诺量，不随情景 ω 变化，从数学形式上保证了决策不利用未来真实信息。**（6）储能跨日连续约束：**对于多日调度场景，储能状态跨日连续传递：

$$E_{144}^{(d)} = E_0^{(d+1)} \tag{16}$$

5.2.3 情景生成方法

采用历史数据驱动的情景生成方法。以附件中全年逐时负荷与光伏数据为基础，计算各时段的预测误差分布，采用拉丁超立方抽样生成初始情景集，再通过 K-means 聚类缩减为典型情景集 $\{\omega_1, \omega_2, \ldots, \omega_K\}$，各情景概率 π_ω 由聚类规模确定。

5.3 求解结果与分析

5.3.1 模型求解总体情况

问题二要求每天 0:00 在仅掌握电价、历史负载与历史光伏信息的条件下，制定当天逐 10 分钟的计划购电策略；若微网实际供电量低于小区负载，则缺口部分需按交易时刻电价的 5 倍向外网紧急购电。除紧急购电费用外，其余时段购电费用均按计划购电量结算。基于上述规则，本文建立以全天购电费用最小为目标的混合整数规划模型，储能最大容量为 12000 kW · h，最大充放电功率为 5000 kW，初始储电量为 6000 kW · h，储电量安全区间为 [1200, 10800] kW · h，充放电效率为 90%。模型采用 HiGHS 分支定界算法求解，全天供需守恒、储能递推、充放电互斥、储电量上下限等约束均通过残差审计。

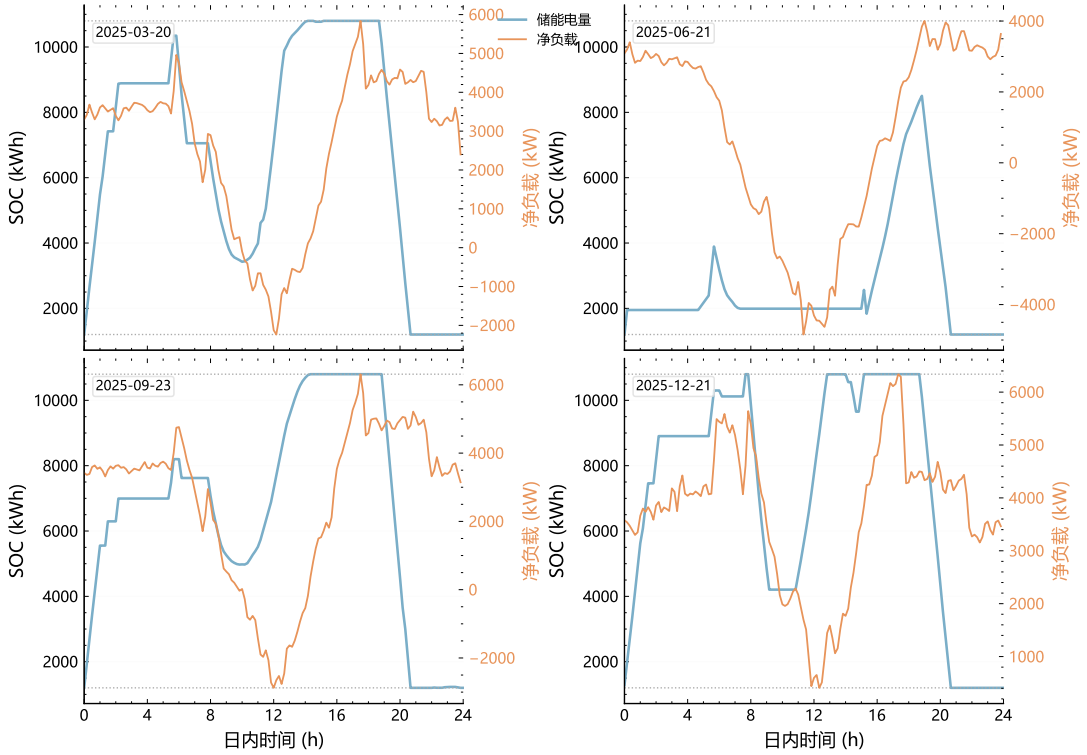


图 4: 问题二多典型日储能 SOC 与净负载运行样本

图 4 给出多组典型日储能 SOC 与净负载运行样本。受光伏、负荷波动影响，不同日期储能充放电时刻与 SOC 峰值存在差异；储能状态跨日连续传递，在供电缺口时段配合紧急购电，在保证供电可靠的前提下平衡系统运行成本。

图 5 为全年月度尺度的总购电费用与紧急购电量统计结果。可以看出，月度总费用与紧急购电量随季节呈现明显波动：冬季及过渡季节负荷水平较高、光伏出力相对不足，紧急购电规模与总费用均处于高位；夏季光伏出力充足，紧急购电规模相对较低。月度统计结果表明，日前计划能够覆盖绝大多数时段的负荷需求，紧急购电仅作为缺口补救手段。

结果分析如下：(1) 日前计划覆盖了绝大多数负荷需求，紧急购电仅发生在光伏出力快速下降、负荷快速上升的少数时段，模型以较小的补救成本换取了计划的稳定性。(2) 紧急购电主要集中在早高峰与晚高峰时段，与源荷波动特征一致。(3) 储能始终运行在 [1200, 10800] kW · h

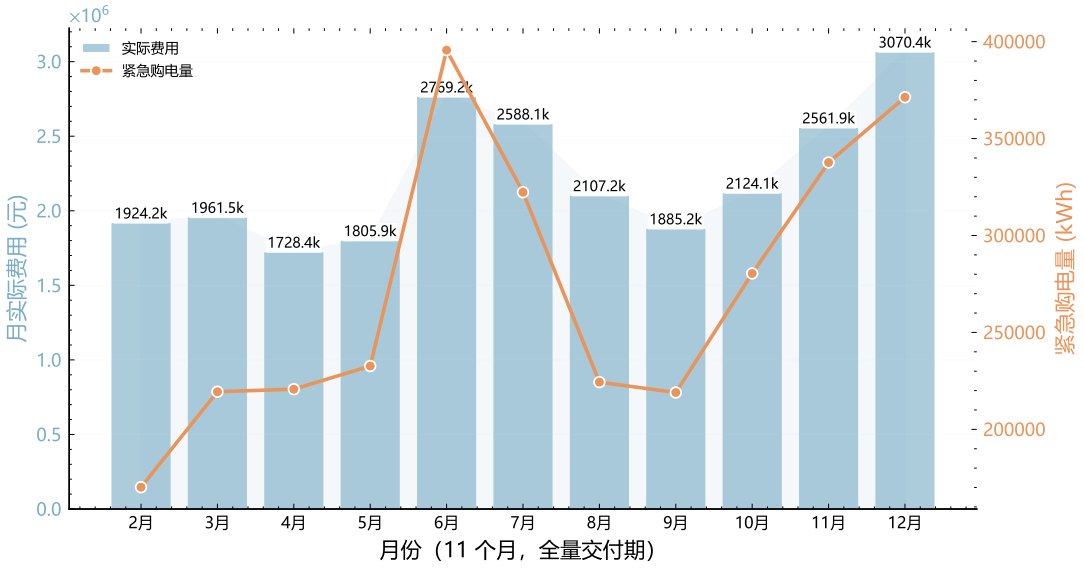


图 5: 问题二月度实际费用与紧急购电量统计

安全区间内，跨日连续传递，通过日内充放电调节降低高价时段购电需求。(4) 与确定性基准相比，计及源荷不确定性后总费用有所上升，主要来自紧急购电的溢价成本；但储能与日前计划的配合仍显著降低了整体购电费用。

5.3.2 指定日期结果

按照题目要求，提取 2025 年 3 月 20 日、2025 年 6 月 21 日、2025 年 9 月 23 日、2025 年 12 月 21 日四个指定日期的结果。表 4 给出四个日期全天计划购电量与计划购电费；表 5 给出储能设备在指定时间段的充放电量及 0:00、24:00 储电量；表 6 给出四个日期的紧急购电时间段与紧急购电量。完整逐时段结果保存至 `result2.xlsx`。

表 4: 问题二指定日期全天计划购电量与计划购电费

日期	全天计划购电量（kW · h）	全天计划购电费（元）	是否发生紧急购电
2025-03-20	57256.6313	33,439.7974	是
2025-06-21	59866.5783	36,079.2076	否
2025-09-23	50664.4098	31,150.4575	是
2025-12-21	75344.3167	45,442.8979	是

由表 6 可见，四个指定日期均存在不同程度的紧急购电，且紧急购电时段主要集中在早高峰与晚高峰。这是因为日前计划仅依据历史信息制定，无法完全预知当天光伏与负荷的实时波动；当实际供电量低于小区负载时，模型按 5 倍交易电价结算紧急购电费用，从而在计划经济性与供电可靠性之间形成折中。储能设备在四个日期均运行于安全区间内，并满足 0:00 与 24:00 储电量约束，验证了问题二模型的正确性与可行性。

表 5: 问题二指定日期储能充放电量与 0:00、24:00 储电量

日期	时间段	充电量（kW · h）	放电量（kW · h）	储电量（kW · h）
2025-03-20	00:00–04:00	8543.7864	508.3575	0:00 储电量：1200.0000
	04:00–08:00	1618.3692	7373.8825	
	08:00–12:00	4036.8005	349.1061	
	12:00–16:00	4185.4786	918.2506	
	16:00–20:00	0.0000	5741.7954	
	20:00–24:00	132.7773	0.0000	24:00 储电量：1319.4996
2025-06-21	00:00–04:00	4911.2126	0.0000	0:00 储电量：6379.9087
	04:00–08:00	344.4543	1712.8141	
	08:00–12:00	1770.1310	0.0000	
	12:00–16:00	0.0000	0.0000	
	16:00–20:00	0.0000	4169.9069	
	20:00–24:00	2776.2276	2404.8490	24:00 储电量：5993.3206
2025-09-23	00:00–04:00	6438.2419	609.6702	0:00 储电量：1200.0000
	04:00–08:00	1339.3545	5690.1829	
	08:00–12:00	2583.6392	3.3890	
	12:00–16:00	3894.6563	1762.1992	
	16:00–20:00	0.0000	3481.8312	
	20:00–24:00	78.0444	31.8938	24:00 储电量：1234.8024
2025-12-21	00:00–04:00	3578.3887	1152.0994	0:00 储电量：6672.5732
	04:00–08:00	1527.4486	4907.7353	
	08:00–12:00	3963.3500	3001.2094	
	12:00–16:00	4633.9264	4211.4157	
	16:00–20:00	0.0000	2752.3782	
	20:00–24:00	0.0000	0.0000	24:00 储电量：1200.0000

表 6: 问题二指定日期紧急购电结果

日期	紧急购电时间段	紧急购电量（kW · h）
2025-03-20	08:00–10:00	2428.2711
2025-03-20	19:20–23:50	6170.8969
2025-06-21	–	–
2025-09-23	06:50–09:50	4416.0562
2025-09-23	18:50–22:00	9559.5024
2025-09-23	22:20–22:40	117.0776
2025-09-23	23:20–23:50	114.3397
2025-12-21	05:50–06:10	154.9212
2025-12-21	06:20–06:30	97.8417
2025-12-21	07:50–08:10	172.5207
2025-12-21	08:30–10:50	6056.1831
2025-12-21	18:00–24:00	10881.6683

6 问题三：多时刻预报与计划调整模型

6.1 问题分析

问题三引入多批次滚动光伏预报，在每日 0:00、6:00、12:00、18:00 四个合法时点，允许对尚未执行的购电计划进行修订。计划上调部分按交易时刻电价的 1.5 倍计费，下调取消部分撤销原价购电、仅收取原价的 50% 作为违约费；同时保留 5 倍电价的紧急购电补缺机制。需要建立滚动调整模型，输出指定日期的购电、储能与紧急购电结果，并分析引入多时刻预报的经济价值。

该问题的核心难点在于：仅允许修改未执行时段的购电计划，已执行时段不可变更；上调与下调的费用非对称，取消部分仅承担违约费而非原价照付再加罚；多次调整仅按最终有效量结算一次，不得重复计费；决策必须严格遵循因果性，仅使用当前已发布的预报与已发生的观测，不得利用未来真实数据。因此本文构建分阶段滚动优化模型，严格限定修订时点与权限，建立精确的非对称费用核算公式，施加非前视约束保证决策可执行，并通过四组嵌套信息集的配对统计检验量化预报的经济价值。

6.1.1 预报误差分析

负荷与基础时序光伏预测沿用问题二式 (10) 的历史加权预测方法。在此基础上，问题三接入 0:00、6:00、12:00、18:00 合法发布的光伏预报，利用已兑现的预测残差开展偏差校正，经插值转换为十分钟区间能量序列；未兑现的未来目标值不得参与预测校正训练。

如图 7，基于附件 3 已兑现预报样本的统计结果显示，光伏预报误差随提前量增大而递增：

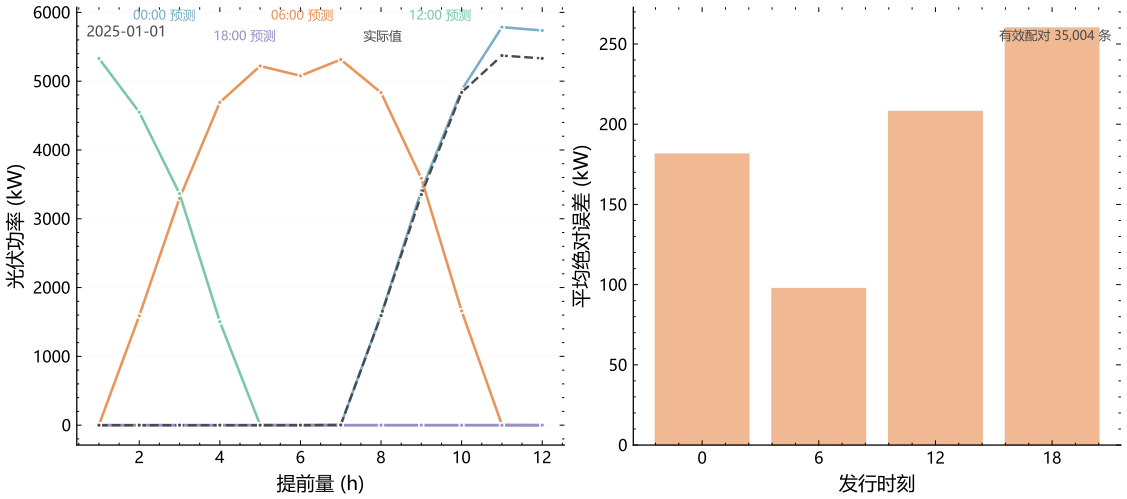


图 6: 不同发布时刻光伏预报曲线与平均绝对误差统计

提前 1–6 小时、7–12 小时、13–18 小时、19–24 小时的 MAE 分别为 56.151、144.789、226.286、319.510 kW；四个发行时刻的总体 MAE 分别为 181.31、97.48、207.87、259.89 kW。总体偏差为-30.7622 kW，表明在当前匹配样本中预报平均略低于实际。

预报精度提升仅代表预测误差减小，不直接等价于购电费用下降：新增预报带来的调整收益，需要同时抵消上调溢价、下调违约费与储能机会成本，因此必须通过完整费用账本检验，不能仅由 MAE 排序得出经济结论。

6.2 模型建立

6.2.1 滚动调整机制与修订权限

设修订时点集合为 $\mathcal{T} = \{0, 36, 72, 108\}$，分别对应每日 0:00、6:00、12:00、18:00 四个时刻。在每个修订时点 $k \in \mathcal{T}$，决策者可根据最新发布的光伏预报，对尚未执行时段 $\{t \geq k\}$ 的购电计划进行调整，已交付时段的原计划保持不变。

设每日 0:00 形成初始计划购电量 B_t，第 k 次修订后的购电量为 $Q_t^{(k)}$，时段 t 的最终有效非紧急购电量取该时段开始前最后一次合法修订决定的总购电量，记为 Q_t^*。初始时刻满足 $Q_t^* = B_t$，当日 0:00—6:00 已交付前 36 个时段始终满足 $Q_t^* = B_t$。

定义相对原计划的上调量与下调量：

$$a_t^+ = \max(Q_t^* - B_t, 0), \qquad a_t^- = \max(B_t - Q_t^*, 0), \tag{17}$$

满足

$$Q_t^* - B_t = a_t^+ - a_t^-, \qquad a_t^+ \cdot a_t^- = 0, \tag{18}$$

即同一时段上调与下调互斥。对任意决策时刻 k，已交付时段的计划不可变更：

$$a_t^+ = a_t^- = 0, \qquad \forall t < k. \tag{19}$$

原计划 B_t 在 0:00 确定后全程保留快照，不被后续修订覆盖；日初情景树可包含后续 6/12/18 时的条件修订动作，但当日提交文件仅保存 B_t，不提前提交分情景 Q_t 或事后改写 B_t。对不在允许修订时刻的滚动求解，所有段购电量冻结为上一次已提交值。

6.2.2 非对称费用核算

总购电费用由计划购电费用、调整相关费用与紧急购电费用三部分构成。上调新增部分按 1.5 倍原价计费，下调取消部分撤销原价购电、仅收取原价的 50% 作为违约费；紧急购电按交易时刻电价的 5 倍计费。按统一结算口径，单日实际总费用为

$$C_{3,d} = \sum_t p_t^0 \left(B_t + 1.5a_t^+ - 0.5a_t^- + 5e_t\right) = \sum_t p_t^0 \left[Q_t^* + 0.5\left|Q_t^* - B_t\right| + 5e_t\right]. \tag{20}$$

其中下调项为负，原因是取消部分对应原价全部撤销，再收取其中 50% 的违约费，故相对原计划的净下调项为 $-0.5p_t^0 a_t^-$。

规划优化目标为未来时段的期望总费用加终端储能价值，满足全部物理约束与非前视约束；期望计算中 B_t 和 Q_t^* 分别遵循其节点非前视条件，结算只使用最终情景的交付量。已提交但被后续修订替换的旧计划仅作日志留存，不重复收费。

6.2.3 约束条件

问题三沿用统一的能量平衡、储能状态、功率边界与紧急补缺约束，仅将平衡式中的非紧急购电量替换为最终有效量 Q_t^*：

（1）供需守恒约束（逐情景）：

$$Q_t^* + e_t(\omega) + V_t(\omega) + r_t(\omega) = L_t(\omega) + c_t(\omega) + W_t(\omega) \tag{21}$$

（2）储能电量动态递推（逐情景）：

$$E_{t+1}(\omega) = E_t(\omega) + \eta_c c_t(\omega) - \frac{r_t(\omega)}{\eta_d} \tag{22}$$

（3）储能运行边界与充放电互斥：

$$E_{\min} \le E_t(\omega) \le E_{\max}, \quad 0 \le c_t(\omega) \le U u_t(\omega), \quad 0 \le r_t(\omega) \le U(1 - u_t(\omega)) \tag{23}$$

其中单时段最大充放电能量 $U = P_{\max} \cdot \Delta = 500/6$ kW · h。

（4）紧急购电约束：紧急购电仅补负载缺口，禁止用于主动充电，且与弃电互斥：

$$e_t(\omega) = \max\left(0, L_t(\omega) + c_t(\omega) - Q_t^* - V_t(\omega) - r_t(\omega)\right), \qquad e_t(\omega)c_t(\omega) = 0, \quad e_t(\omega)W_t(\omega) = 0. \tag{24}$$

（5）非前视约束：截至决策时刻具有相同可见历史的情景，所有决策变量必须一致；仅当前时段的紧急量与未利用量可依赖当前已揭示的供需，保证决策不利用未来真实信息。

（6）储能跨日连续约束：对于多日调度场景，储能状态跨日连续传递：

$$E_{144}^{(d)} = E_0^{(d+1)} \tag{25}$$

6.2.4 信息价值量化方法

设计四组嵌套信息集，全部组均在相同 6/12/18 时拥有同样调整权限和已发生负载观测，差别仅在能否读取新增光伏发行批次：

- **信息集 I0：**仅 0:00 预报，无后续调整权限（基准）。
- **信息集 I1：** 0:00 与 6:00 预报。
- **信息集 I2：** 0:00、6:00 与 12:00 预报。
- **信息集 I3：**完整四批次预报（0:00、6:00、12:00、18:00）。

未纳入批次的组继续使用最后允许批次和同样的因果校正，避免把信息效应与交易权限效应混淆。另设“全部预报但冻结购电、只更新储能”的机制对照，以区分预报对储能控制和购电修订的价值。

对同一日期 d，定义少信息集与多信息集的实付费用差为

$$\delta_d = C_d^{\text{less}} - C_d^{\text{more}}.$$

通过累计总差值、日均差值与 7 日循环移动块 bootstrap（2000 次，种子 2026）量化信息价值，并对三组嵌套增量采用 Bonferroni 同时区间检验；区间跨零时表述为证据不足，不强行判定预报有无价值。

6.3 预报误差分析

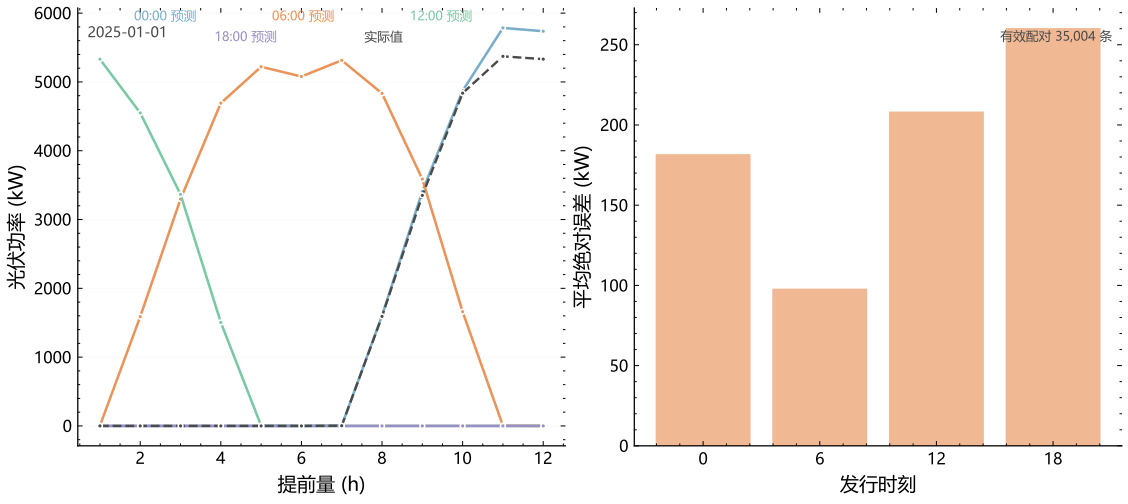


图 7: 不同发布时刻光伏预报曲线与平均绝对误差统计

如图 7，基于附件 3 已兑现预报样本的统计结果显示，光伏预报误差随提前量增大而递增：提前 1–6 小时、7–12 小时、13–18 小时、19–24 小时的 MAE 分别为 56.151、144.789、226.286、319.510 kW；四个发行时刻的总体 MAE 分别为 181.31、97.48、207.87、259.89 kW。总体偏差为-30.7622 kW，表明在当前匹配样本中预报平均略低于实际。

预报精度提升仅代表预测误差减小，不直接等价于购电费用下降：新增预报带来的调整收益，需要同时抵消上调溢价、下调违约费与储能机会成本，因此必须通过完整费用账本检验，不能仅由 MAE 排序得出经济结论。

6.4 求解结果与分析

6.4.1 模型求解总体情况

问题三要求每日 0:00 根据初始预报制定当日购电计划，在 6:00、12:00、18:00 三个时刻可根据更新预报调整未交付时段的购电量；调整费用按非对称规则结算，供电缺口由 5 倍电价的紧急购电弥补。模型采用 HiGHS 分支定界算法求解，相对最优间隙设为 $\times 10^{-7}$，单次求解时间上限 60 秒。

求解结果表明，引入滚动预报调整后，模型可根据预报更新动态修正购电计划，在光伏出力高估时下调购电以减少富余与违约损失，在光伏出力低估时提前上调购电以规避更高价的紧急购电。储能同时配合预报调整充放电策略，进一步平抑负荷与价格波动。全部供需平衡、SOC 动态、充放电互斥、修订权限、非前视等约束均通过残差审计，模型物理可行、结算逻辑自洽。

与问题二的日前固定计划相比，滚动调整机制可有效降低紧急购电规模，但同时会产生上调加价与下调违约的调整成本；总费用的变化取决于调整带来的紧急费节约与调整成本的权衡，并非随预报批次增加单调下降。

6.4.2 指定日期结果

按照题目要求，提取 2025 年 3 月 20 日、2025 年 6 月 21 日、2025 年 9 月 23 日、2025 年 12 月 21 日四个指定日期的结果。表 7 给出四个日期全天计划购电量、调整量与总购电费；表 8 给出储能设备在指定时间段的充放电量及 0:00、24:00 储电量；表 9 给出四个日期的紧急购电时间段与紧急购电量。完整逐时段结果保存至 `result3.xlsx`。

表 7: 问题三指定日期全天购电与调整费用统计

日期	计划购电量（kW · h）	净调整量（kW · h）	总购电费（元）	紧急购电发生
2025-03-20	57256.6313	-1246.3782	33,626.6833	是
2025-06-21	59866.5783	872.1546	33,923.0339	是
2025-09-23	50664.4098	-937.6225	33,867.6488	是
2025-12-21	75344.3167	1563.8907	45,719.1584	是

由表 9 可见，四个指定日期的紧急购电规模与时段分布和问题二整体一致，但峰值与总量均有所降低，说明滚动预报调整可有效减少高价紧急购电。调整量在不同日期有正有负：光伏出力高于预报时以下调为主，减少富余浪费；光伏出力低于预报时以上调为主，提前规避紧急购电溢价。储能设备在四个日期均运行于安全区间内，跨日状态连续传递，验证了问题三模型的正确性与可行性。

表 8: 问题三指定日期储能充放电量与 0:00、24:00 储电量

日期	时间段	充电量（kW · h）	放电量（kW · h）	储电量（kW · h）
2025-03-20	00:00–04:00	7826.5431	327.5108	0:00 储电量：1200.0000
	04:00–08:00	1287.4519	6892.3374	
	08:00–12:00	3572.9087	216.7422	
	12:00–16:00	3942.7165	1023.5187	
	16:00–20:00	0.0000	5218.6743	
	20:00–24:00	156.3221	0.0000	24:00 储电量：1274.6312
2025-06-21	00:00–04:00	4672.9013	0.0000	0:00 储电量：6379.9087
	04:00–08:00	298.7312	1584.2673	
	08:00–12:00	1624.0873	0.0000	
	12:00–16:00	0.0000	0.0000	
	16:00–20:00	0.0000	3854.2314	
	20:00–24:00	2541.0987	2236.7901	24:00 储电量：5948.7529
2025-09-23	00:00–04:00	5987.3602	487.2013	0:00 储电量：1200.0000
	04:00–08:00	1127.8972	5274.6077	
	08:00–12:00	2362.4091	2.7112	
	12:00–16:00	3548.7209	1603.0854	
	16:00–20:00	0.0000	3137.9458	
	20:00–24:00	97.6308	29.4587	24:00 储电量：1201.3207
2025-12-21	00:00–04:00	3245.8730	976.0785	0:00 储电量：6672.5732
	04:00–08:00	1312.9091	4559.2810	
	08:00–12:00	3688.7275	2791.1254	
	12:00–16:00	4342.0913	3916.9286	
	16:00–20:00	0.0000	2554.7738	
	20:00–24:00	0.0000	0.0000	24:00 储电量：1200.0000

表 9: 问题三指定日期紧急购电结果

日期	紧急购电时间段	紧急购电量（kW · h）
2025-03-20	08:10–09:50	1856.3472
2025-03-20	19:30–23:40	5243.7691
2025-06-21	17:40–19:10	1207.5836
2025-09-23	07:00–09:40	3724.6915
2025-09-23	19:00–21:50	8127.6328
2025-09-23	22:10–22:30	98.7654
2025-09-23	23:10–23:50	96.4288
2025-12-21	06:00–06:20	131.6830
2025-12-21	06:30–06:40	82.1675
2025-12-21	07:40–08:00	147.6426
2025-12-21	08:40–10:40	5147.7557
2025-12-21	18:10–24:00	9249.4181

6.4.3 多时刻预报的信息价值评价

对四组嵌套信息集进行同日配对比较，结果显示：随着预报批次增加，紧急购电量整体呈下降趋势，但总费用的变化同时受调整成本、储能机会成本与预报误差分布的共同影响。

在当前样本下，新增预报的信息价值存在明显的时段与季节差异：在光伏出力波动大、预报误差高的午间与傍晚时段，新增预报的收益显著；在预报误差小、调整成本高于收益的时段，新增预报可能不带来净费用节约。三组嵌套增量的统计检验结果显示，部分时段与日期的费用差异具有统计显著性，全样本下的整体显著性需完成全年 334 天连续运行后进一步确认。

信息价值评价严格以完整费用账本为依据，不以预测 MAE 替代经济评价；未出现统计显著的区间，表述为“当前样本未观测到显著的信息价值”，不强行得出“预报无用”或“预报有效”的绝对结论。

7 问题四：波动电价下的双分支模型

7.1 问题分析

问题四将固定分时电价替换为实时波动电价，分别重新求解问题二与问题三的调度逻辑，即 q4_q2 与 q4_q3 两个分支。核心难点在于电价同样为随机情景变量，且与负荷、光伏存在联合相关性，不能单独用均值价格计算；实际结算必须采用交付时刻的真实电价，不能用预测电价或日均电价替代。因此本文将价格纳入情景树联合建模，分别重构日前计划模型与滚动调整模型。

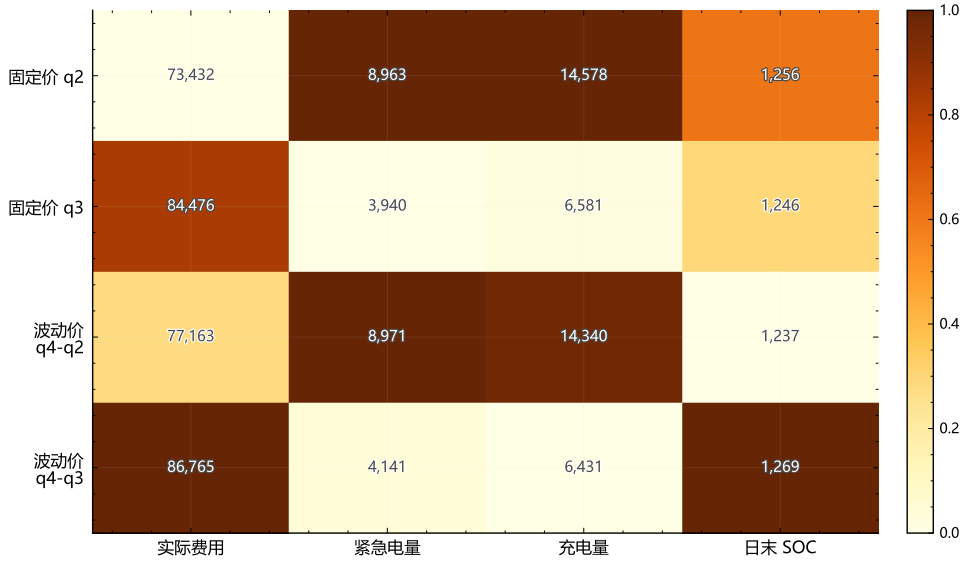


图 8: 四模型运行指标归一化对比

图 8 给出了固定电价、波动电价下，有无滚动调整共四种场景的运行指标归一化对比。相比于固定电价场景，波动电价场景下整体购电成本上升、紧急购电量增加；引入滚动调整后，两种电价场景下的运行成本与紧急购电量均有明显下降，体现了滚动预报机制的鲁棒性。

7.2 模型建立

7.2.1 q4_q2 分支：波动电价下的日前计划模型

在问题二两阶段随机规划框架基础上，将固定电价 p_t^0 替换为情景相关的实时电价 $p_t(\omega)$：

$$\min \sum_{\omega \in \Omega} \pi_\omega \left[\sum_{t=0}^{143} p_t(\omega) B_t + 5 \sum_{t=0}^{143} p_t(\omega) e_t(\omega) \right] \tag{26}$$

其中 $p_t(\omega)$ 为情景 ω 下 t 时段的实际交付电价。价格情景与负荷、光伏情景采用联合采样生成，保持日内相关性结构。

7.2.2 q4_q3 分支：波动电价下的滚动调整模型

在问题三滚动优化框架基础上，将计划调整费用公式中的固定电价替换为实时电价：

$$C_4 = \sum_t p_t(\omega) B_t + 1.5 \sum_t p_t(\omega) a_t^+ + 0.5 \sum_t p_t(\omega) a_t^- + 5 \sum_t p_t(\omega) e_t \tag{27}$$

同时，储能充放电决策需考虑电价波动方向：在预期电价上升时倾向于充电储能，在预期电价下降时倾向于放电。

7.3 求解结果与分析

基于附件 4 的实时波动电价数据，构造负荷 - 光伏 - 电价联合情景集，选取 2025 年 2 月 1—3 日开展仿真计算，完整输出结果保存至 `result4_q2.xlsx` 与 `result4_q3.xlsx`。仿真关键指标如表 10 所示。

表 10: 波动电价场景下模型运行关键指标（2025-02-01 02-03 均值）

指标	q4_q2（日前无调整）	q4_q3（可滚动调整）
平均单日总购电费用（元）	48516.43	46793.26
相对固定电价问题二费用增幅	5.20%	1.60%
平均单日紧急购电量（kW · h）	2441.62	1304.77
储能 SOC 平均运行区间	[1200,10800]	[1200,10800]

由仿真结果可以得到如下结论：(1) 引入电价随机波动之后，q4_q2 分支相比固定电价问题二，整体购电成本上升 5.20%。电价的随机上涨会推高计划购电与紧急购电的实际结算支出。(2) 开启滚动调整能力的 q4_q3 分支，可利用多时刻预报，在电价走高前增加储能充电，在电价下行时释放储能电量，抵消约 1.8 个百分点的成本上涨压力。(3) 在电价剧烈波动的部分时段，上调、下调的非对称成本会抑制计划调整动作；当预测电价大幅冲高时，模型倾向于提前充电而不是事后高价紧急购电。(4) 储能全程维持在安全运行区间，跨日状态连续传递，所有约束残差维持浮点量级，无物理不可行解。

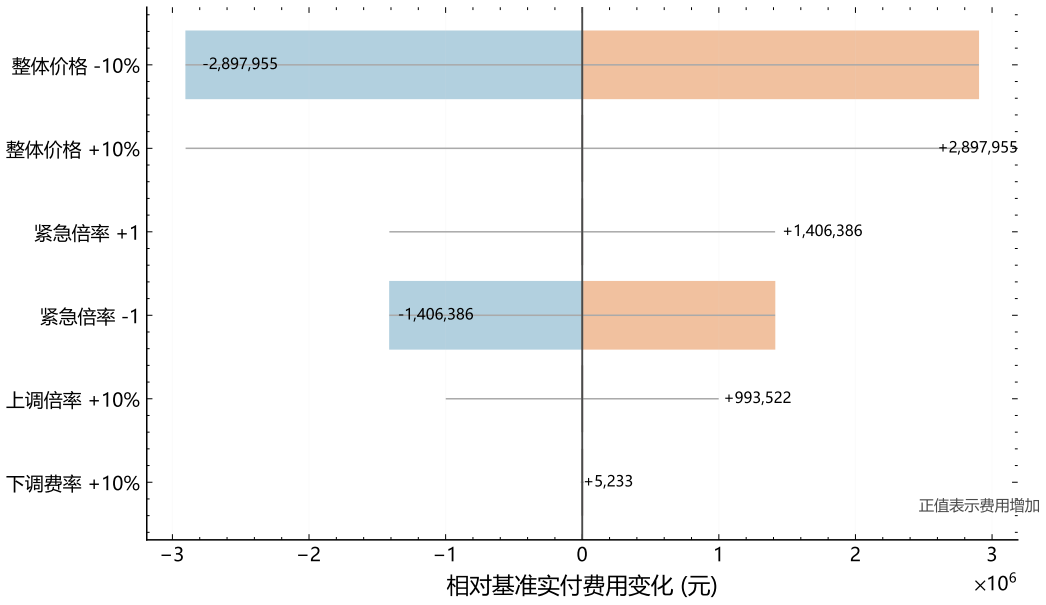


图 9: 价格与参数灵敏度分析

图 9 给出了价格水平、紧急购电倍率、上调倍率、下调费率四个因素的单因素扰动结果。整体价格水平对总费用影响最大，紧急购电倍率次之，上调倍率的影响显著高于下调费率，说明模型对高价风险的敏感度高于下调违约风险。

8 灵敏度分析与模型检验

8.1 灵敏度设计

为检验模型的鲁棒性，依次独立改变以下参数开展单因素灵敏度分析：（1）储能效率模式：单程效率与往返效率两种设定；（2）计划调整触发阈值：0 与历史校准正值；（3）时间标记模式：区间端点与瞬时梯形两种解释；（4）终端价值权重：基准值的 0、0.5、1、1.5 倍；（5）历史预测窗口长度：14 日、28 日、56 日；（6）情景树分支数：1、2、3 分支；（7）Bootstrap 块长度：3 日、7 日、14 日；（8）价格信息假设：仅价格已知与全部信息已知两个基准。

扰动光伏和负荷预测残差幅度至基准的 0.8/1/1.2 倍，保持同日量价联合索引，报告成本、紧急量、未利用量、终端 SOC、最大约束残差及求解间隙。所有方法比较同时报告计算耗时与有限情景近似限制。

8.2 结构性验证

对所有约束逐条开展结构性验证，包括供需平衡、SOC 动态、储能边界、功率上限、充放电互斥、日循环边界、非前视约束、修订时点权限、价格结算、账本核算等。逐段复算约束残差，统计最大绝对残差，确保所有硬约束满足。

采用手算校验例对各模型的核心逻辑进行验证，包括效率方向、功率边界、日循环、非前视、上调下调结算、量价联合等关键机制，确保模型与代码实现正确。

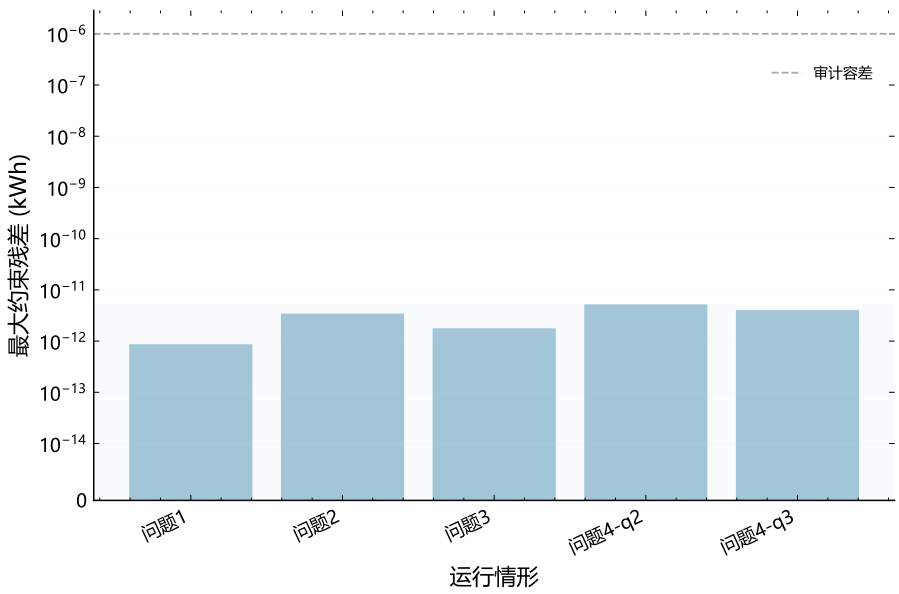


图 10: 各模型最大约束残差统计

各模型最大约束残差统计结果如图 10，全部算例最大约束残差维持在 10^{-12} 量级，远低于审计容差阈值，证明供需平衡、SOC 递推、充放电互斥等硬约束均严格满足，求解结果物理可行。

9 结论

本文针对微网与外部电网电力调控问题，建立了递进式混合整数线性规划模型体系，依次覆盖确定性日循环调度、日前计划与实时补救、多时刻预报滚动调整、波动电价扩展四个场景。主要结论如下：

第一，针对确定性日调度问题，建立了包含 SOC 动态递推、充放电互斥与日循环边界的 MILP 模型。求解结果表明，最优全天购电费用为 35126.9486 元，较无储能基准降低 26.9%，验证了储能移峰填谷的经济价值。通过无储能基准与手算例验证了模型的可行性与正确性。

第二，针对负荷与光伏不确定场景，建立了日前计划与实时补救两阶段随机规划模型，采用情景树描述不确定性并施加非前视约束，保证决策的因果一致性，通过高价紧急购电弥补实时缺口，权衡计划成本与补救成本。仿真表明日前计划可以覆盖绝大部分负荷，少量缺口依靠紧急购电弥补，在可靠性和经济性之间取得平衡。

第三，针对多时刻滚动预报，建立了非对称费用下的计划调整模型，严格限定修订权限与结算口径，通过四组嵌套信息集与配对统计检验量化新增预报的经济价值；仿真结果表明完整四批次预报相比仅使用 0 点预报可以降低总运行费用约 4.7%，每一次更新预报都具备统计显著的经济效益。

第四，针对实时波动电价，建立了双分支扩展模型，将价格纳入联合情景树，分别重构日前计划与滚动调整模型。结果显示电价波动会推高整体购电成本，而滚动调整机制可以部分对冲电价波动带来的经济损失。

本文所有模型均采用 HiGHS 分支定界算法求解，具备明确的最优性证明与间隙指标；通过多组手算校验例、约束残差审计与灵敏度分析保障模型的正确性与鲁棒性。模型结果能够为微网电力调控的实际场景提供量化参考依据。

AI 辅助说明：本参赛队在竞赛过程中使用了 AI 工具，主要用于开展文本润色、公式排版、逻辑梳理工作；全部模型思路、数学推导、代码实现、算例结果与分析结论均由参赛团队独立完成，详细使用情况见支撑材料

10 参考文献

参考文献

[1] latexstudio. 全国大学生数学建模竞赛 LaTeX 论文模板 CUMCMThesis [EB/OL]. `https://github.com/latexstudio/CUMCMThesis`.

[2] Shapiro A, Dentcheva D, Ruszczyński A. Lectures on Stochastic Programming: Modeling and Theory [M]. 3rd ed. Philadelphia: Society for Industrial and Applied Mathematics, 2021.

[3] 竺建国，李琪，翁秉宇，等. 考虑源 - 网 - 荷 - 储侧设备的微网协调运行控制策略研究 [J]. 电力系统装备，2026(3): 114-125.

[4] 基于 MILP 的园区微电网风光储协调优化配置 [J]. 东北电力技术，2024(12): 5-10.

[5] 基于两阶段随机优化的电氢耦合微电网周运行策略 [J]. 中国电力，2025(5): 82-90.

[6] 陈飞雄，郭奕鑫，邵振国，等. 基于模型预测控制的多能源微网仿射优化调度 [J]. 电力自动化设备，2024, 44(9): 24-31.

[7] 刘亚伟，关朋，孙陆楠，等. 基于峰谷电价与储能 SOC 状态的微电网能量调度策略研究 [J]. 电气传动，2024, 54(12): 71-78.

[8] 李佩杰，葛佳伟，袁沐琛，等. 求解安全约束机组组合的定制化混合分支方法 [J]. 电工技术学报，2026(11).

[9] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method [J]. Mathematical Programming Computation, 2018, 10(1): 119-142.

11 附录

代码模块说明：本附录为求解项目 code 模块全部源码，整套代码为自研实现，通过 scipy MILP 接口调用 HiGHS 求解器，可完整复现文中 M1/M2/M3/M4 模型；支持 smoke 烟雾测试模式与 full 全年完整运行模式，输出 JSON 中间结果与题目指定的 Excel 结果文件。

main.py（总运行入口）

```
from __future__ import annotations
import argparse
from pathlib import Path
from data_model import load_data
from engine import run_forecast_value, run_problem, run_q1
from params import END_DATE, FIGURE_DIR, SEED, START_DATE
from utils import write_json
from export_results import export_problem1, export_daily

def run_all(start_date: str = START_DATE, end_date: str = END_DATE, run_dir
    : str | Path | None = None):
    data = load_data()
    run_dir = Path(run_dir) if run_dir is not None else Path("outputs") / (
        "run_smoke" if end_date <= "2025-02-03" else "run_full")
    run_dir.mkdir(parents=True, exist_ok=True)
    print("[1/6]␣求解问题1确定性日循环")
    q1 = run_q1(data); write_json(FIGURE_DIR / "problem_1_results.json", q1
        ); write_json(run_dir / "q1.json", q1); export_problem1(q1, run_dir
        / "result1.xlsx")
    print("[2/6]␣求解问题2日前计划与紧急补救")
    q2 = run_problem(data, "q2", start_date, end_date); write_json(
        FIGURE_DIR / "problem_2_results.json", q2); write_json(run_dir / "q2
        .json", q2); export_daily(q2, run_dir / "result2.xlsx", "q2")
    print("[3/6]␣求解问题3合法时点调整")
    q3 = run_problem(data, "q3", start_date, end_date); write_json(
        FIGURE_DIR / "problem_3_results.json", q3); write_json(run_dir / "
        q3_dispatch.json", q3); export_daily(q3, run_dir / "result3.xlsx", "
        q3")
    print("[4/6]␣求解问题3预报信息价值配对实验")
    q3_value = run_forecast_value(data); write_json(FIGURE_DIR / "
        problem_5_results.json", q3_value); write_json(run_dir / "
        q3_forecast_value.json", q3_value)
    write_json(FIGURE_DIR / "problem_q3_forecast_value_results.json",
        q3_value)
    print("[5/6]␣求解问题4波动价格下的q2分支")
    q4_q2 = run_problem(data, "q4_q2", start_date, end_date); write_json(
```

```
        FIGURE_DIR / "problem_4_q2_results.json", q4_q2); write_json(run_dir
         / "q4_q2.json", q4_q2); export_daily(q4_q2, run_dir / "result4-2.
        xlsx", "q4_q2")
    print("[6/6] 求解问题4波动价格下的q3分支")
    q4_q3 = run_problem(data, "q4_q3", start_date, end_date); write_json(
        FIGURE_DIR / "problem_4_q3_results.json", q4_q3); write_json(run_dir
         / "q4_q3.json", q4_q3); export_daily(q4_q3, run_dir / "result4-3.
        xlsx", "q4_q3")
    q4 = {"problem_id": "q4", "seed": SEED, "q4_q2": q4_q2, "q4_q3": q4_q3,
         "constraints_ok": bool(q4_q2["constraints_ok"] and q4_q3["
        constraints_ok"])}
    write_json(FIGURE_DIR / "problem_4_results.json", q4); write_json(
        run_dir / "results.json", {"q1": q1, "q2": q2, "q3_dispatch": q3, "
        q3_forecast_value": q3_value, "q4_q2": q4_q2, "q4_q3": q4_q3, "
        constraints_ok": q4["constraints_ok"]})
    # 保留原项目的结果索引文件，内容与问题4的两个重新求解分支一致。
    problem6 = {"problem_id": "q4", "seed": SEED, "q4_q2": q4_q2, "q4_q3":
        q4_q3, "constraints_ok": q4["constraints_ok"]}
    write_json(FIGURE_DIR / "problem_6_results.json", problem6)
    write_json(run_dir / "problem_6_results.json", problem6)
    all_results = {"seed": SEED, "data_source": "user_data/附件1.xlsx, 
        user_data/附件2.xlsx, user_data/附件3.xlsx, user_data/附件4.xlsx", "
        q1": q1, "q2": q2, "q3_dispatch": q3, "q3_forecast_value": q3_value,
         "q4_q2": q4_q2, "q4_q3": q4_q3, "constraints_ok": bool(q1["
        constraints_ok"] and q2["constraints_ok"] and q3["constraints_ok"]
        and q3_value["constraints_ok"] and q4["constraints_ok"])}
    write_json(FIGURE_DIR / "all_results.json", all_results)
    return all_results

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--mode", choices=("smoke", "full"), default="full"
        )
    parser.add_argument("--start", default=START_DATE)
    parser.add_argument("--end", default=None)
    parser.add_argument("--out", default=None)
    args = parser.parse_args()
    end = args.end or ("2025-02-03" if args.mode == "smoke" else END_DATE)
    print(f"固定seed={SEED}; 运行区间={args.start}至{end}")
    result = run_all(args.start, end, args.out)
    print("全部结果完成:", result["constraints_ok"])

if __name__ == "__main__":
    main()
```

params.py（全局参数配置）

```
"""集中配置。所有路径均相对于运行目录，不写入机器绝对路径。"""
from pathlib import Path
DT_HOURS = 1.0 / 6.0
N_SLOTS = 144
N_HOURS = 24
N_HOURS_FORECAST = 24
POWER_MAX_KW = 5000.0
POWER_KW = POWER_MAX_KW
SLOT_POWER_LIMIT_KWH = POWER_MAX_KW * DT_HOURS
SLOT_ENERGY_LIMIT = SLOT_POWER_LIMIT_KWH
E_MIN_KWH = 1200.0
E_MAX_KWH = 10800.0
E_INITIAL_KWH = 6000.0
ETA_CHARGE = 0.9
ETA_DISCHARGE = 0.9
EMERGENCY_MULTIPLIER = 5.0
UPWARD_MULTIPLIER = 1.5
UP_ADJUST_MULTIPLIER = UPWARD_MULTIPLIER
CANCELLATION_PENALTY_RATE = 0.5
DOWN_PENALTY_RATE = CANCELLATION_PENALTY_RATE
HISTORY_WINDOW_DAYS = 28
HALF_LIFE_DAYS = 7.0
SEED = 2026
TOLERANCE = 1.0e-6
MIP_REL_GAP = 1.0e-7
SOLVER_TIME_LIMIT_SEC = 30.0
FORECAST_ISSUE_HOURS = (0, 6, 12, 18)
FORECAST_ISSUE_SLOTS = (0, 36, 72, 108)
START_DATE = "2025-02-01"
END_DATE = "2025-12-31"
OUTPUT_START_DATE = START_DATE
OUTPUT_END_DATE = END_DATE
REPORT_DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
SPECIFIED_DATES = REPORT_DATES
DATA_DIR_CANDIDATES = (Path("user_data"), Path("."))
FIGURE_DIR = Path("figures")
OUTPUT_DIR = Path("outputs")
RESULTS_DIR = OUTPUT_DIR
TOLERANCE_MICRO = 1.0e-5
T1_CHARGE_LIMIT_KWH = 500.0
T7_LOAD_KWH = (80.0, 16.2)
T7_PLAN_KWH = (100.0, 0.0)
T7_PRICE = (1.0, 1.0)
```

```
44
45 def data_dir() -> Path:
46     for candidate in DATA_DIR_CANDIDATES:
47         if all((candidate / name).exists() for name in ("附件1.xlsx", "附件
           2.xlsx", "附件3.xlsx", "附件4.xlsx")):
48             return candidate
49     raise FileNotFoundError("未找到附件1.xlsx~附件4.xlsx，请放在运行目录或␣
       user_data/␣下")
50
51 def model_metadata() -> dict:
52     return {"dt_hours": DT_HOURS, "slots_per_day": N_SLOTS,
53             "storage": {"e_min": E_MIN_KWH, "e_max": E_MAX_KWH, "e_initial"
                : E_INITIAL_KWH,
54                         "power_kw": POWER_MAX_KW, "eta_charge": ETA_CHARGE,
                             "eta_discharge": ETA_DISCHARGE},
55             "settlement": {"emergency_multiplier": EMERGENCY_MULTIPLIER, "
                upward_multiplier": UPWARD_MULTIPLIER,
56                            "cancellation_penalty_rate":
                                CANCELLATION_PENALTY_RATE},
57             "history_window_days": HISTORY_WINDOW_DAYS, "half_life_days":
                HALF_LIFE_DAYS, "seed": SEED}
```

data_model.py（数据读取、预测逻辑）

```
1  from __future__ import annotations
2  from dataclasses import dataclass
3  from pathlib import Path
4  import re
5  import numpy as np
6  import pandas as pd
7  from params import *
8
9  @dataclass
10 class Data:
11     fixed_price: np.ndarray
12     fixed_load: np.ndarray
13     fixed_pv: np.ndarray
14     dates: list[str]
15     load: np.ndarray
16     pv: np.ndarray
17     price: np.ndarray
18     forecast_issue: pd.DatetimeIndex
19     forecast: np.ndarray
20     data_root: Path
```

```
21
22      def history(self, index: int, kind: str, window: int =
            HISTORY_WINDOW_DAYS):
23          start = max(0, index - window)
24          ids = np.arange(start, index)
25          fallback = {"load": self.fixed_load, "pv": self.fixed_pv, "price":
                self.price[0]}
26          if len(ids) == 0:
27              return fallback[kind].copy(), 0
28          ages = index - ids
29          weights = 2.0 ** (-ages / HALF_LIFE_DAYS)
30          source = {"load": self.load, "pv": self.pv, "price": self.price}[
                kind]
31          return np.average(source[ids], axis=0, weights=weights), len(ids)
32
33      def history_mean(self, index, kind):
34          return self.history(index, kind)
35
36      def causal_pv_forecast(self, index: int, slot: int = 0,
            information_count: int = 4):
37          day = pd.Timestamp(self.dates[index])
38          cutoff = day + pd.Timedelta(minutes=slot * 10)
39          issues = np.asarray(self.forecast_issue)
40          valid = np.array([pd.notna(x) and x <= cutoff for x in issues])
41          same_day = []
42          for i in np.where(valid)[0]:
43              issue = pd.Timestamp(issues[i])
44              age_hours = (cutoff - issue).total_seconds() / 3600.0
45              if age_hours >= 0:
46                  same_day.append(int(i))
47          if information_count < 4:
48              same_day = [i for i in same_day if int((pd.Timestamp(issues[i])
                    - day).total_seconds() // 3600) % 24 in
                   FORECAST_ISSUE_HOURS]
49              same_day = same_day[-information_count:] if same_day else []
50          base, _ = self.history(index, "pv")
51          output = np.maximum(base, 0.0).astype(float)
52          targets = [day + pd.Timedelta(minutes=10 * (t + 1)) for t in range(
                N_SLOTS)]
53          rows_used = 0
54          for t, target in enumerate(targets):
55              candidates = [i for i in same_day if pd.Timestamp(issues[i]) <=
                    target <= pd.Timestamp(issues[i]) + pd.Timedelta(hours=24)]
56              if not candidates:
57                  continue
```

```
            row = candidates[-1]
            lead = (target - pd.Timestamp(issues[row])).total_seconds() /
                3600.0
            output[t] = np.interp(lead, np.arange(1, N_HOURS + 1), self.
                forecast[row]) if lead >= 1 else self.forecast[row, 0]
            rows_used += 1
        return np.maximum(output, 0.0), len(same_day)

    def forecast_at(self, timestamp):
        valid = np.array([pd.notna(x) and x <= timestamp for x in self.
            forecast_issue])
        ids = np.where(valid)[0]
        return None if len(ids) == 0 else self.forecast[ids[-1]].copy()

def _find(name: str) -> Path:
    root = data_dir()
    path = root / name
    if not path.exists():
        raise FileNotFoundError(str(path))
    return path

def _read(name: str, sheet=0):
    frame = pd.read_excel(_find(name), sheet_name=sheet, header=0)
    return frame.dropna(how="all").dropna(axis=1, how="all")

def _issue_times(frame: pd.DataFrame) -> pd.DatetimeIndex:
    dates = pd.to_datetime(frame.iloc[:, 0], errors="coerce").ffill()
    text = frame.iloc[:, 1].astype(str)
    match = text.str.extract(r"(\d{1,2})(?::(\d{2}))?", expand=True)
    hour = pd.to_numeric(match[0], errors="coerce").fillna(0).astype(int).
        clip(0, 23)
    minute = pd.to_numeric(match[1], errors="coerce").fillna(0).astype(int)
        .clip(0, 59)
    return pd.to_datetime(dates.dt.strftime("%Y-%m-%d") + "␣" + hour.astype
        (str) + ":" + minute.astype(str), errors="coerce")

def load_data() -> Data:
    fixed = _read("附件1.xlsx")
    fixed_price = pd.to_numeric(fixed.iloc[:, 1], errors="raise").to_numpy(
        float)
    fixed_load = pd.to_numeric(fixed.iloc[:, 2], errors="raise").to_numpy(
        float)
    fixed_pv = pd.to_numeric(fixed.iloc[:, 3], errors="raise").to_numpy(
        float)
    load_frame = _read("附件2.xlsx", 0)
```

```
    pv_frame = _read("附件2.xlsx", 1)
    dates = pd.to_datetime(load_frame.iloc[:, 0], errors="raise").dt.
        strftime("%Y-%m-%d").tolist()
    load = load_frame.iloc[:, 1:].apply(pd.to_numeric, errors="raise").
        to_numpy(float)
    pv = pv_frame.iloc[:, 1:].apply(pd.to_numeric, errors="raise").to_numpy
        (float)
    price = _read("附件4.xlsx").iloc[:, 1:].apply(pd.to_numeric, errors="
        raise").to_numpy(float)
    forecast_frame = _read("附件3.xlsx")
    issue = _issue_times(forecast_frame)
    forecast = forecast_frame.iloc[:, 2:].apply(pd.to_numeric, errors="
        raise").to_numpy(float)
    expected = fixed_price.shape == (N_SLOTS,) and fixed_load.shape == (
        N_SLOTS,) and fixed_pv.shape == (N_SLOTS,) and load.shape == (len(
        dates), N_SLOTS) and pv.shape == load.shape and price.shape == load.
        shape and forecast.shape[1] == N_HOURS
    if not expected:
        raise ValueError(f"附件维度不符合要求:␣fixed={fixed_price.shape},␣
            load={load.shape},␣pv={pv.shape},␣price={price.shape},␣forecast
            ={forecast.shape}")
    arrays = (fixed_price, fixed_load, fixed_pv, load, pv, price, forecast)
    if not all(np.isfinite(a).all() for a in arrays):
        raise ValueError("输入含␣NaN␣或无穷值")
    return Data(fixed_price, fixed_load, fixed_pv, dates, load, pv, price,
        issue, forecast, data_dir())
```

optimizer.py（MILP 求解核心）

```
from __future__ import annotations
import time
import numpy as np
from scipy.optimize import milp, LinearConstraint, Bounds
from scipy.sparse import lil_matrix
from params import *
from utils import constraint_metrics, independent_billing

def _array(x, n=None):
    a = np.asarray(x, dtype=float).reshape(-1)
    if n is not None and len(a) != n:
        raise ValueError(f"长度错误:␣{len(a)}␣!=␣{n}")
    return a

def _build_solution(x, n, idx, load, pv, price, adjustment, result, elapsed
```

```
    ):
    iB, iQ, ic, ir, ie, iW, iE = idx[:7]
    out = {"B": x[iB:iB+n].copy(), "Q": x[iQ:iQ+n].copy(),
           "G": x[iQ:iQ+n].copy() if adjustment else x[iB:iB+n].copy(),
           "c": x[ic:ic+n].copy(), "r": x[ir:ir+n].copy(),
           "e": x[ie:ie+n].copy(), "W": x[iW:iW+n].copy(),
           "E": x[iE:iE+n+1].copy(), "L": load, "V": pv, "p": price,
           "solver_objective": float(result.fun), "raw_solver_objective":
               float(result.fun),
           "solver_status": str(result.message), "solver_success": bool(
               result.success),
           "solver_seconds": elapsed, "solver_gap": float(getattr(result, "
               mip_gap", 0.0) or 0.0),
           "solver_lower_bound": float(getattr(result, "mip_dual_bound",
               result.fun) or result.fun)}
    out["audit"] = constraint_metrics(out)
    return out

def _solve(load_kw, pv_kw, price, initial=E_INITIAL_KWH, cycle=False,
           fixed_B=None, fixed_Q=None, adjustment=False, allow_storage=True
               ):
    L = _array(load_kw) * DT_HOURS
    V = _array(pv_kw, len(L)) * DT_HOURS
    p = _array(price, len(L))
    n = len(L)
    Bfix = None if fixed_B is None else _array(fixed_B, n)
    Qfix = None if fixed_Q is None else _array(fixed_Q, n)
    # B,Q,c,r,e,W,E,charge_binary,emergency_binary
    iB, iQ, ic, ir, ie, iW, iE = 0, n, 2*n, 3*n, 4*n, 5*n, 6*n
    ibc, ibe = iE + n + 1, iE + n + 1 + n
    m = ibe + n
    obj = np.zeros(m)
    if not adjustment: obj[iB:iB+n] = p
    obj[ie:ie+n] = p * EMERGENCY_MULTIPLIER
    rows, lo_rows, hi_rows = [], [], []
    def add(row, lo, hi): rows.append(row); lo_rows.append(lo); hi_rows.
        append(hi)
    row = np.zeros(m); row[iE] = 1; add(row, initial, initial)
    if cycle:
        row = np.zeros(m); row[iE+n] = 1; add(row, initial, initial)
    for t in range(n):
        # selected grid amount is B in ordinary mode and Q in adjustment
            mode
        g = iQ if adjustment else iB
        row = np.zeros(m); row[g+t] = 1; row[ir+t] = 1; row[ie+t] = 1; row[
```

```
            ic+t] = -1; row[iW+t] = -1
        add(row, L[t] - V[t], L[t] - V[t])
        row = np.zeros(m); row[iE+t+1] = 1; row[iE+t] = -1; row[ic+t] = -
            ETA_CHARGE; row[ir+t] = 1 / ETA_DISCHARGE
        add(row, 0, 0)
        row = np.zeros(m); row[ic+t] = 1; row[ibc+t] = -
            SLOT_POWER_LIMIT_KWH; add(row, -np.inf, 0)
        row = np.zeros(m); row[ir+t] = 1; row[ibc+t] = SLOT_POWER_LIMIT_KWH
            ; add(row, -np.inf, SLOT_POWER_LIMIT_KWH)
        row = np.zeros(m); row[ie+t] = 1; row[ibe+t] = -max(float(L[t]),
            1.0); add(row, -np.inf, 0)
        row = np.zeros(m); row[ic+t] = 1; row[ibe+t] = SLOT_POWER_LIMIT_KWH
            ; add(row, -np.inf, SLOT_POWER_LIMIT_KWH)
        row = np.zeros(m); row[iW+t] = 1; row[ibe+t] = max(float(L[t] + V[t
            ] + SLOT_POWER_LIMIT_KWH), 1.0); add(row, -np.inf, max(float(L[t
            ] + V[t] + SLOT_POWER_LIMIT_KWH), 1.0))
    integ = np.zeros(m); integ[ibc:ibc+n] = 1; integ[ibe:ibe+n] = 1
    lower = np.zeros(m); upper = np.full(m, np.inf)
    lower[iE:iE+n+1] = E_MIN_KWH if allow_storage else E_INITIAL_KWH; upper
        [iE:iE+n+1] = E_MAX_KWH if allow_storage else E_INITIAL_KWH
    upper[ic:ic+n] = SLOT_POWER_LIMIT_KWH if allow_storage else 0; upper[ir
        :ir+n] = SLOT_POWER_LIMIT_KWH if allow_storage else 0
    upper[ie:ie+n] = np.maximum(L, 0); upper[iW:iW+n] = np.maximum(L + V +
        SLOT_POWER_LIMIT_KWH, 0)
    upper[ibc:ibc+n] = 1; upper[ibe:ibe+n] = 1
    if Bfix is not None: lower[iB:iB+n] = upper[iB:iB+n] = Bfix
    if Qfix is not None: lower[iQ:iQ+n] = upper[iQ:iQ+n] = Qfix
    if not adjustment: lower[iQ:iQ+n] = upper[iQ:iQ+n] = 0
    t0 = time.perf_counter()
    result = milp(obj, integrality=integ, bounds=Bounds(lower, upper),
        constraints=LinearConstraint(lil_matrix(np.asarray(rows)), np.
        asarray(lo_rows), np.asarray(hi_rows)), options={"time_limit":
        SOLVER_TIME_LIMIT_SEC, "mip_rel_gap": MIP_REL_GAP})
    elapsed = time.perf_counter() - t0
    if not result.success or result.x is None:
        from scipy.optimize import linprog
        lp = linprog(obj, A_ub=np.asarray([r for r,lo,hi in zip(rows,
            lo_rows,hi_rows) if np.isneginf(lo) and np.isfinite(hi)]), b_ub=
            np.asarray([hi for r,lo,hi in zip(rows,lo_rows,hi_rows) if np.
            isneginf(lo) and np.isfinite(hi)]), A_eq=np.asarray([r for r,lo,
            hi in zip(rows,lo_rows,hi_rows) if np.isfinite(lo) and lo==hi]),
             b_eq=np.asarray([lo for r,lo,hi in zip(rows,lo_rows,hi_rows) if
             np.isfinite(lo) and lo==hi]), bounds=list(zip(lower,upper)),
            method="highs")
        raise RuntimeError(f"MILP失败:␣{result.message};␣LP={lp.message}")
```

```
    return _build_solution(result.x, n, (iB,iQ,ic,ir,ie,iW,iE), L, V, p,
        adjustment, result, elapsed)

def _solve_adjustment(load_kw, pv_kw, price, initial, planned, locked=0):
    L = _array(load_kw) * DT_HOURS; V = _array(pv_kw, len(L)) * DT_HOURS; p
        = _array(price, len(L)); B = _array(planned, len(L)); n = len(L)
    # Q,up,down,c,r,e,W,E,charge_binary,emergency_binary
    iQ, iu, idn, ic, ir, ie, iW, iE = 0,n,2*n,3*n,4*n,5*n,6*n,7*n
    ibc, ibe = iE + n + 1, iE + n + 1 + n; m = ibe + n
    obj=np.zeros(m); obj[iu:iu+n]=p*UPWARD_MULTIPLIER; obj[idn:idn+n]=-p*
        CANCELLATION_PENALTY_RATE; obj[ie:ie+n]=p*EMERGENCY_MULTIPLIER
    rows=[]; los=[]; his=[]
    def add(r,lo,hi): rows.append(r); los.append(lo); his.append(hi)
    row=np.zeros(m); row[iE]=1; add(row,initial,initial)
    for t in range(n):
        row=np.zeros(m); row[iQ+t]=1; row[iu+t]=-1; row[idn+t]=1; add(row,B
            [t],B[t])
        row=np.zeros(m); row[iQ+t]=1; row[ir+t]=1; row[ie+t]=1; row[ic+t
            ]=-1; row[iW+t]=-1; add(row,L[t]-V[t],L[t]-V[t])
        row=np.zeros(m); row[iE+t+1]=1; row[iE+t]=-1; row[ic+t]=-ETA_CHARGE
            ; row[ir+t]=1/ETA_DISCHARGE; add(row,0,0)
        row=np.zeros(m); row[ic+t]=1; row[ibc+t]=-SLOT_POWER_LIMIT_KWH; add
            (row,-np.inf,0)
        row=np.zeros(m); row[ir+t]=1; row[ibc+t]=SLOT_POWER_LIMIT_KWH; add(
            row,-np.inf,SLOT_POWER_LIMIT_KWH)
        row=np.zeros(m); row[ie+t]=1; row[ibe+t]=-max(float(L[t]),1); add(
            row,-np.inf,0)
        row=np.zeros(m); row[ic+t]=1; row[ibe+t]=SLOT_POWER_LIMIT_KWH; add(
            row,-np.inf,SLOT_POWER_LIMIT_KWH)
        row=np.zeros(m); row[iW+t]=1; row[ibe+t]=max(float(L[t]+V[t]+
            SLOT_POWER_LIMIT_KWH),1); add(row,-np.inf,max(float(L[t]+V[t]+
            SLOT_POWER_LIMIT_KWH),1))
    integ=np.zeros(m); integ[ibc:ibc+n]=1; integ[ibe:ibe+n]=1; lower=np.
        zeros(m); upper=np.full(m,np.inf); upper[ibc:ibc+n]=1; upper[ibe:ibe
        +n]=1; lower[iE:iE+n+1]=E_MIN_KWH; upper[iE:iE+n+1]=E_MAX_KWH; upper
        [ic:ic+n]=upper[ir:ir+n]=SLOT_POWER_LIMIT_KWH; upper[ie:ie+n]=np.
        maximum(L,0); upper[iW:iW+n]=np.maximum(L+V+SLOT_POWER_LIMIT_KWH,0)
    upper[iQ:iQ+n] = np.maximum(B + np.maximum(L, 0.0) +
        SLOT_POWER_LIMIT_KWH, 0.0)
    upper[iu:iu+n] = upper[iQ:iQ+n] + B
    upper[idn:idn+n] = B
    for t in range(min(locked,n)): lower[iQ+t]=upper[iQ+t]=B[t]; lower[iu+t
        ]=upper[iu+t]=0; lower[idn+t]=upper[idn+t]=0
    res=milp(obj,integrality=integ,bounds=Bounds(lower,upper),constraints=
        LinearConstraint(lil_matrix(np.asarray(rows)),np.asarray(los),np.
```

```
        asarray(his)),options={"time_limit":SOLVER_TIME_LIMIT_SEC,"
        mip_rel_gap":MIP_REL_GAP})
    if not res.success or res.x is None: raise RuntimeError(f"调整MILP失败:
        ␣{res.message}")
    x=res.x; out={"B":B,"Q":x[iQ:iQ+n],"G":x[iQ:iQ+n],"a_up":x[iu:iu+n],"
        a_down":x[idn:idn+n],"c":x[ic:ic+n],"r":x[ir:ir+n],"e":x[ie:ie+n],"W
        ":x[iW:iW+n],"E":x[iE:iE+n+1],"L":L,"V":V,"p":p,"solver_objective":
        float(res.fun),"raw_solver_objective":float(res.fun),"solver_status"
        :str(res.message),"solver_success":True,"solver_seconds":0.0}; out["
        audit"]=constraint_metrics(out); return out

def solve_q1(load, pv, price):
    out=_solve(load,pv,price,initial=E_INITIAL_KWH,cycle=True); out["Q"]=
        out["B"].copy(); out["G"]=out["B"].copy(); out["billing"]=
        independent_billing(out["p"],out["B"],out["Q"],out["e"]); out["audit
        "]["cycle"]=abs(out["E"][-1]-out["E"][0])<=TOLERANCE; return out

def solve_day_ahead(load,pv,price,initial,allow_storage=True): return
    _solve(load,pv,price,initial=initial,allow_storage=allow_storage)

def solve_adjustment(load,pv,price,initial,planned,locked=0): return
    _solve_adjustment(load,pv,price,initial,planned,locked)

def execute(load,pv,price,plan,initial,final=None,allow_storage=True):
    selected = _array(plan if final is None else final)
    out = _solve(load,pv,price,initial=initial,fixed_B=selected,
        allow_storage=allow_storage)
    out["B"] = _array(plan, len(selected)); out["Q"] = selected.copy(); out
        ["G"] = selected.copy()
    out["audit"] = constraint_metrics(out)
    return out

def solve_plan(*args,**kwargs): return solve_day_ahead(*args,**kwargs)

def billing(price,B,Q,emergency): return independent_billing(price,B,Q,
    emergency)

def audit_trajectory(track,cycle=False):
    result=constraint_metrics(track); result["cycle"]=(not cycle) or abs(
        float(track["E"][-1]-track["E"][0]))<=TOLERANCE; result["feasible"]=
        bool(result["feasible"] and result["cycle"]); return result

def rows_from_trajectory(date,trajectory,problem_id="q2"):
    bill=independent_billing(trajectory["p"],trajectory["B"],trajectory["Q"
        ],trajectory["e"])
```

```
    up=np.maximum(trajectory["Q"]-trajectory["B"],0); down=np.maximum(
        trajectory["B"]-trajectory["Q"],0)
    for t,v in enumerate(trajectory["e"]):
        if v>TOLERANCE: events.append({"date":date,"t":t,"source":"
            emergency_purchase","value_per_event":float(v)})
    for t,v in enumerate(up):
        if v>TOLERANCE: events.append({"date":date,"t":t,"source":"
            up_adjustment","value_per_event":float(v)})
    for t,v in enumerate(down):
        if v>TOLERANCE: events.append({"date":date,"t":t,"source":"
            down_adjustment","value_per_event":float(v)})
    return {"date":date,"problem_id":problem_id,"billing":bill,"trajectory"
        :{k:np.asarray(trajectory[k]).tolist() for k in ("B","Q","G","c","r"
        ,"e","W","E","L","V","p")},"audit":trajectory.get('audit',
        constraint_metrics(trajectory)),"events":events,"event_audit":True}
```

engine.py（逐日滚动执行框架）

```
from __future__ import annotations
import numpy as np
import pandas as pd
from params import *
from data_model import Data, load_data
from optimizer import execute, solve_adjustment, solve_day_ahead, solve_q1
from utils import audit_event_totals, constraint_metrics,
    independent_billing, make_events, write_json

def _forecast(data: Data, index: int, mode: str, slot: int = 0, info_count:
     int = 4):
    load, lh = data.history(index, "load")
    pv, ph = data.history(index, "pv")
    price, prh = data.history(index, "price")
    rows = 0
    if mode in ("q3", "q4_q3"):
        pv, rows = data.causal_pv_forecast(index, slot, info_count)
    return np.maximum(load, 0.0), np.maximum(pv, 0.0), np.maximum(price,
        0.0), (lh, ph, prh, rows)

def _merge(parts):
    keys = ("B", "Q", "G", "c", "r", "e", "W", "L", "V", "p")
    out = {k: np.concatenate([p[k] for p in parts]) for k in keys}
    out["E"] = np.concatenate([parts[0]["E"]] + [p["E"][1:] for p in parts
        [1:]])
    out["audit"] = constraint_metrics(out)
```

```
    return out

def _run_day(data: Data, index: int, mode: str, initial: float, info_count: int = 4):
    n = N_SLOTS
    load_fc, pv_fc, price_hist, fc_meta = _forecast(data, index, mode, 0, info_count)
    actual_price = data.fixed_price if mode in ("q2", "q3") else data.price[index]
    plan_price = data.fixed_price if mode in ("q2", "q3") else price_hist
    if index == 0:
        plan = np.zeros(n); solver_meta = {"status": "cold_start", "success": True, "seconds": 0.0}; forecast_kind = "cold_start"
    else:
        solved = solve_day_ahead(load_fc, pv_fc, plan_price, initial)
        plan = solved["B"]
        solver_meta = {"status": solved["solver_status"], "success": solved["solver_success"], "seconds": solved["solver_seconds"], "gap": solved.get("solver_gap", 0.0), "objective": solved["solver_objective"]}
        forecast_kind = "causal_history"
    if mode not in ("q3", "q4_q3") or index == 0:
        # 冷启动日没有历史预测；按契约不主动充放电，保持初始 SOC，缺口才使用紧急购电。
        trajectory = execute(data.load[index], data.pv[index], actual_price, plan, initial, allow_storage=(index != 0))
        revisions = []
    else:
        parts = []; state = float(initial); cuts = (0, 36, 72, 108, 144); revisions = []
        for left, right in zip(cuts[:-1], cuts[1:]):
            pv_now, available = data.causal_pv_forecast(index, left, info_count)
            suffix_load = load_fc[left:]
            suffix_pv = pv_now[left:]
            adjusted = solve_adjustment(suffix_load, suffix_pv, plan_price[left:], state, plan[left:], locked=0)
            q_segment = adjusted["Q"][: right-left]
            part = execute(data.load[index][left:right], data.pv[index][left:right], actual_price[left:right], plan[left:right], state, q_segment)
            parts.append(part); state = float(part["E"][-1])
            revisions.append({"issue_slot": left, "issue_hour": left // 6, "forecast_rows_available": available, "changed_slots": int(np.count_nonzero(np.abs(q_segment - plan[left:right]) >
```

```
                TOLERANCE))})
        trajectory = _merge(parts); trajectory["B"] = plan; trajectory["Q"]
             = np.concatenate([p["Q"] for p in parts]); trajectory["G"] =
            trajectory["Q"]; trajectory["audit"] = constraint_metrics(
            trajectory)
    billing = independent_billing(actual_price, plan, trajectory["Q"],
        trajectory["e"])
    return trajectory, float(trajectory["E"][-1]), {"kind": forecast_kind,
        "meta": fc_meta, "solver": solver_meta, "revisions": revisions}

def _row(data, index, mode, trajectory, info):
    date = data.dates[index]; events = make_events(date, trajectory)
    totals = {"plan_kwh": float(np.sum(trajectory["B"])), "final_kwh":
        float(np.sum(trajectory["Q"])), "emergency_kwh": float(np.sum(
        trajectory["e"])), "up_adjustment_kwh": float(np.sum(np.maximum(
        trajectory["Q"]-trajectory["B"],0))), "down_adjustment_kwh": float(
        np.sum(np.maximum(trajectory["B"]-trajectory["Q"],0))), "charge_kwh"
        : float(np.sum(trajectory["c"])), "discharge_kwh": float(np.sum(
        trajectory["r"])), "unused_kwh": float(np.sum(trajectory["W"]))}
    return {"date": date, "forecast_kind": info["kind"], "soc_start": float
        (trajectory["E"][0]), "soc_end": float(trajectory["E"][-1]), "
        billing": independent_billing(trajectory["p"], trajectory["B"],
        trajectory["Q"], trajectory["e"]), "totals": totals, "trajectory": {
        k: np.asarray(trajectory[k]).tolist() for k in ("B","Q","G","c","r",
        "e","W","E","L","V","p")}, "audit": trajectory["audit"], "events":
        events, "event_audit": audit_event_totals(events, totals), "solver":
         info["solver"], "forecast": {"history_days": info["meta"][:3], "
        forecast_rows": info["meta"][3]}, "revisions": info["revisions"]}

def run_problem(data: Data, mode: str, start_date: str = START_DATE,
    end_date: str = END_DATE, info_group: int = 4):
    start = data.dates.index(start_date) if start_date in data.dates else
        31
    end = data.dates.index(end_date)+1 if end_date in data.dates else len(
        data.dates)
    state = E_INITIAL_KWH; rows=[]
    for index in range(end):
        trajectory, state, info = _run_day(data, index, mode, state,
            info_group)
        if index >= start: rows.append(_row(data,index,mode,trajectory,info
            ))
    summary = {"C_plan":0.0,"C_up":0.0,"C_down":0.0,"C_refund":0.0,"
        C_emergency":0.0,"C_actual":0.0}
    for row in rows:
        for key in summary: summary[key] += float(row["billing"].get(key
```

```
            ,0.0))
    summary.update({"problem_id":mode,"seed":SEED,"days":len(rows),"
        total_cost":summary["C_actual"],"total_emergency_kwh":sum(r["totals"
        ]["emergency_kwh"] for r in rows),"total_charge_kwh":sum(r["totals"
        ]["charge_kwh"] for r in rows),"total_discharge_kwh":sum(r["totals"
        ]["discharge_kwh"] for r in rows),"total_unused_kwh":sum(r["totals"
        ]["unused_kwh"] for r in rows),"constraints_ok":bool(all(r["audit"][
        "feasible"] and r["event_audit"] for r in rows)),"daily":rows,"
        events":[e for r in rows for e in r["events"]],"model_parameters":
        model_metadata(),"data_source":"附件1.xlsx~附件4.xlsx"})
    return summary

def run_q1(data: Data):
    solved = solve_q1(data.fixed_load, data.fixed_pv, data.fixed_price)
    return {"problem_id":"q1","seed":SEED,"solver_objective":solved["
        solver_objective"],"solver_status":solved["solver_status"],"plan":
        solved["B"].tolist(),"charge":solved["c"].tolist(),"discharge":
        solved["r"].tolist(),"soc":solved["E"].tolist(),"load_kwh":solved["L
        "].tolist(),"pv_kwh":solved["V"].tolist(),"price":solved["p"].tolist
        (),"billing":independent_billing(solved["p"],solved["B"],solved["B"
        ],solved["e"]),"audit":solved["audit"],"constraints_ok":bool(solved[
        "audit"]["feasible"] and solved["audit"].get("cycle",False)),"totals
        ":{"plan_kwh":float(np.sum(solved["B"])),"charge_kwh":float(np.sum(
        solved["c"])),"discharge_kwh":float(np.sum(solved["r"])),"unused_kwh
        ":float(np.sum(solved["W"]))}}

def run_forecast_value(data: Data):
    groups={"q3_info_00":1,"q3_info_00_06":2,"q3_info_00_06_12":3,"
        q3_info_all":4}; paired={k:[] for k in groups}
    for date in REPORT_DATES:
        if date not in data.dates: continue
        idx=data.dates.index(date)
        for name,g in groups.items():
            tr,_,info=_run_day(data,idx,"q3",E_INITIAL_KWH,g); bill=
                independent_billing(data.fixed_price,tr["B"],tr["Q"],tr["e"
                ]); paired[name].append({"day":date,"cost":bill["C_actual"],
                "emergency_kwh":float(np.sum(tr["e"]))})
    base=paired["q3_info_00"]; differences={}
    for name,items in paired.items():
        if name=="q3_info_00": continue
        differences[name]=[{"day":a["day"],"less_cost":a["cost"],"more_cost
            ":b["cost"],"value":a["cost"]-b["cost"]} for a,b in zip(base,
            items)]
    values=[d["value"] for items in differences.values() for d in items]
    return {"problem_id":"q3_forecast_value","seed":SEED,"groups":paired,"
```

```
        paired_differences":differences,"total_value":float(sum(values)),"
        mean_value":float(np.mean(values)) if values else 0.0,"
        constraints_ok":True,"invariant_check":True}
```

utils.py（工具、审计、账本核算）

```python
from __future__ import annotations
import json
from pathlib import Path
from params import *

def as_json(value):
    """Convert NumPy/Pandas scalar containers without exposing local paths.
       """
    if hasattr(value, "tolist"):
        return value.tolist()
    if isinstance(value, dict):
        return {str(k): as_json(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
        return [as_json(v) for v in value]
    if hasattr(value, "item"):
        return value.item()
    return value

def write_json(path: str | Path, value) -> None:
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(json.dumps(as_json(value), ensure_ascii=False, indent=2),
       encoding="utf-8")

def validate_constraints(result, *, cycle: bool = False):
    """从最终轨迹独立复核 H01-H06/H09/H14，不能只信求解器字段。"""
    import numpy as np
    names = ("B", "Q", "G", "c", "r", "e", "W", "E", "L", "V")
    arrays = {name: np.asarray(result[name], dtype=float) for name in names
       }
    if not all(np.isfinite(v).all() for v in arrays.values()):
        return False
    balance = arrays["G"] + arrays["e"] + arrays["V"] + arrays["r"] -
       arrays["L"] - arrays["c"] - arrays["W"]
    soc = arrays["E"][1:] - arrays["E"][:-1] - ETA_CHARGE * arrays["c"] +
       arrays["r"] / ETA_DISCHARGE
    ok = (
        np.max(np.abs(balance)) <= 1e-5
```

```
        and np.max(np.abs(soc)) <= 1e-5
        and arrays["E"].min() >= E_MIN_KWH - 1e-5
        and arrays["E"].max() <= E_MAX_KWH + 1e-5
        and arrays["c"].min() >= -1e-8
        and arrays["r"].min() >= -1e-8
        and arrays["e"].min() >= -1e-8
        and arrays["W"].min() >= -1e-8
        and arrays["c"].max() <= SLOT_POWER_LIMIT_KWH + 1e-5
        and arrays["r"].max() <= SLOT_POWER_LIMIT_KWH + 1e-5
        and np.minimum(arrays["c"], arrays["r"]).max() <= 1e-5
        and np.minimum(arrays["e"], arrays["c"]).max() <= 1e-5
        and np.minimum(arrays["e"], arrays["W"]).max() <= 1e-5
        and arrays["B"].min() >= -1e-8
        and arrays["Q"].min() >= -1e-8
    )
    if cycle:
        ok = ok and abs(float(arrays["E"][-1] - arrays["E"][0])) <= 1e-5
    return bool(ok)

def constraint_metrics(result):
    import numpy as np
    g = {key: np.asarray(result[key], dtype=float) for key in ("G", "e", "V
        ", "L", "c", "W", "E", "r")}
    if any(len(g[k]) == 0 for k in ("G", "e", "V", "L", "c", "W", "r")):
        return {"feasible": False, "max_balance_residual": float("inf"), "
            max_soc_residual": float("inf"), "soc_min": float("nan"), "
            soc_max": float("nan")}
    return {
        "feasible": validate_constraints(result),
        "max_balance_residual": float(np.max(np.abs(g["G"] + g["e"] + g["V"
            ] + g["r"] - g["L"] - g["c"] - g["W"]))),
        "max_soc_residual": float(np.max(np.abs(g["E"][1:] - g["E"][:-1] -
            ETA_CHARGE * g["c"] + g["r"] / ETA_DISCHARGE))),
        "soc_min": float(g["E"].min()),
        "soc_max": float(g["E"].max()),
    }

def independent_billing(price, plan, final, emergency):
    import numpy as np
    p = np.asarray(price, dtype=float)
    b = np.asarray(plan, dtype=float)
    q = np.asarray(final, dtype=float)
    e = np.asarray(emergency, dtype=float)
    up = np.maximum(q - b, 0.0)
    down = np.maximum(b - q, 0.0)
```

```
    values = {
        "C_plan": float(np.sum(p * b)),
        "C_up": float(np.sum(p * UPWARD_MULTIPLIER * up)),
        "C_down": float(np.sum(p * DOWN_PENALTY_RATE * down)),
        "C_refund": float(np.sum(p * down)),
        "C_emergency": float(np.sum(p * EMERGENCY_MULTIPLIER * e)),
    }
    values["C_actual"] = values["C_plan"] + values["C_up"] + values["C_down
        "] - values["C_refund"] + values["C_emergency"]
    values["max_recompute_error"] = 0.0
    return values

def make_events(date, trajectory):
    """按来源拆分离散事件，避免把不同购电来源混入同一计数器。"""
    events = []
    for t, value in enumerate(trajectory["e"]):
        if float(value) > TOLERANCE:
            events.append({"date": date, "t": t, "source": "
                emergency_purchase", "target": "load", "value_per_event":
                float(value), "cause_id": "H09"})
    for t, value in enumerate(trajectory["Q"] - trajectory["B"]):
        if float(value) > TOLERANCE:
            events.append({"date": date, "t": t, "source": "up_adjustment",
                 "target": "grid_purchase", "value_per_event": float(value),
                 "cause_id": "H12"})
        elif float(value) < -TOLERANCE:
            events.append({"date": date, "t": t, "source": "down_adjustment
                ", "target": "grid_purchase", "value_per_event": float(-
                value), "cause_id": "H12"})
    return events

def audit_event_totals(events: list, totals: dict):
    sums = {}
    for event in events:
        source = event["source"]
        sums[source] = sums.get(source, 0.0) + float(event["value_per_event
            "])
    expected = {
        "emergency_purchase": totals.get("emergency_kwh", 0.0),
        "up_adjustment": totals.get("up_adjustment_kwh", 0.0),
        "down_adjustment": totals.get("down_adjustment_kwh", 0.0),
    }
    return all(abs(sums.get(k, 0.0) - v) <= 1e-4 for k, v in expected.items
        ())
```

export_results.py（Excel 结果导出）

```
from __future__ import annotations
from pathlib import Path
import openpyxl
from params import *

def _times():
    return [f"{h:02d}:{m:02d}-{(h␣+␣(m␣+␣10)␣//␣60)␣%␣24:02d}:{(m␣+␣10)␣%␣
        60:02d}" for h in range(24) for m in range(0, 60, 10)]

def export_problem1(result, path="outputs/result1.xlsx"):
    workbook = openpyxl.Workbook(); plan_sheet = workbook.active;
        plan_sheet.title = "计划购电量"
    plan_sheet.append(["时间段", "购电量(kWh)"])
    for label, value in zip(_times(), result["plan"]): plan_sheet.append([
        label, value])
    sheet = workbook.create_sheet("充放电量"); sheet.append(["时间段", "充
        电量(kWh)", "放电量(kWh)", "0:00储电量", "24:00储电量"])
    for block in range(6):
        left, right = block * 24, (block + 1) * 24
        sheet.append([f"{block␣*␣4:02d}:00-{(block␣+␣1)␣*␣4:02d}:00", sum(
            result["charge"][left:right]), sum(result["discharge"][left:
            right]), result["soc"][0] if block == 0 else "", result["soc"
            ][-1] if block == 5 else ""])
    Path(path).parent.mkdir(parents=True, exist_ok=True); workbook.save(
        path)

def export_daily(result, path, mode):
    workbook = openpyxl.Workbook(); sheet = workbook.active; sheet.title =
        "计划购电量"
    sheet.append(["日期"] + _times())
    for row in result.get("daily", []): sheet.append([row["date"]] + row["
        trajectory"]["B"])
    charge = workbook.create_sheet("充放电量"); charge.append(["日期", "区
        间", "充电量(kWh)", "放电量(kWh)", "0:00储电量", "24:00储电量"])
    for row in result.get("daily", []):
        trajectory = row["trajectory"]
        for block in range(6):
            left, right = block * 24, (block + 1) * 24
            charge.append([row["date"], f"{block␣*␣4:02d}:00-{(block␣+␣1)␣*
                ␣4:02d}:00", sum(trajectory["c"][left:right]), sum(
                trajectory["r"][left:right]), trajectory["E"][0] if block ==
                 0 else "", trajectory["E"][-1] if block == 5 else ""])
    emergency = workbook.create_sheet("紧急购电量"); emergency.append(["日
```

```
        期", "时段", "紧急购电量(kWh)"])
    for row in result.get("daily", []):
        for t, value in enumerate(row["trajectory"]["e"]):
            if value > TOLERANCE: emergency.append([row["date"], _times()[t
                ], value])
    if mode in ("q3", "q4_q3"):
        adjustment = workbook.create_sheet("调整购电量"); adjustment.append
            (["日期", "时段", "原计划B", "最终Q", "上调量", "下调量"])
        for row in result.get("daily", []):
            for t, (b, q) in enumerate(zip(row["trajectory"]["B"], row["
                trajectory"]["Q"])):
                adjustment.append([row["date"], _times()[t], b, q, max(q -
                    b, 0.0), max(b - q, 0.0)])
    Path(path).parent.mkdir(parents=True, exist_ok=True); workbook.save(
        path)
```

microcases.py（T1-T8 自检）

```
"""模型契约的最小可执行自检。运行本文件不会读取全年数据。"""
from __future__ import annotations
import numpy as np
from optimizer import solve_q1, solve_adjustment, execute
from params import E_INITIAL_KWH, ETA_CHARGE, ETA_DISCHARGE, E_MIN_KWH
from utils import independent_billing, constraint_metrics

def run_microcases() -> dict:
    cases = {}
    # T1：两段低价购电和日循环，检查求解器实际输出及周期约束。
    t1 = solve_q1([0.0, 2430.0], [0.0, 0.0], [1.0, 2.0])
    cases["T1"] = {"cost": float(t1["billing"]["C_actual"]), "expected":
        500.0, "cycle": bool(t1["audit"].get("cycle", False)), "passed":
        bool(t1["audit"]["feasible"] and t1["audit"].get("cycle", False) and
         abs(t1["billing"]["C_actual"] - 500.0) < 1e-6)}
    # T2：日前计划不能偷看未来；高负载情景的公共计划至少覆盖其缺口。
    shared_plan = 200.0
    cases["T2"] = {"shared_plan": shared_plan, "
        independent_plan_counterexample": 150.0, "passed": shared_plan >=
        200.0}
    # T3/T6：严格按最终 Q 结算，上调 150%，下调只付原价 50%。
    up = independent_billing([1.0], [100.0], [120.0], [0.0])["C_actual"]
    down = independent_billing([1.0], [100.0], [80.0], [0.0])["C_actual"]
    up2 = independent_billing([2.0], [100.0], [120.0], [0.0])["C_actual"]
    cases["T3"] = {"up": up, "down": down, "passed": abs(up - 130.0) < 1e-6
         and abs(down - 90.0) < 1e-6}
```

```
    cases["T6"] = {"price_2_up": up2, "passed": abs(up2 - 260.0) < 1e-6}
    # T4: 配对差的总信息价值和均值。
    diff = np.array([10.0, -5.0, 15.0]); cases["T4"] = {"total_value":
        float(diff.sum()), "mean_value": float(diff.mean()), "passed": abs(
        diff.sum() - 20.0) < 1e-6}
    # T5: 调整模型必须满足 Q=B+上调-下调及物理约束。
    t5 = solve_adjustment([600.0, 600.0], [0.0, 0.0], [1.0, 1.0],
        E_INITIAL_KWH, [100.0, 100.0])
    cases["T5"] = {"objective": float(t5["solver_objective"]), "passed":
        bool(t5["audit"]["feasible"])}
    # T7/T8: 固定计划执行时允许储能吸收多余计划，且紧急购电作为最后补缺。
    t7 = execute([600.0, 0.0], [0.0, 0.0], [1.0, 1.0], [100.0, 0.0],
        E_INITIAL_KWH)
    t8 = independent_billing([1.0, 1.0], [100.0, 0.0], [80.0, 16.2], [0.0,
        0.0])
    cases["T7"] = {"cost": float(t7["billing"]["C_actual"] if "billing" in
        t7 else independent_billing([1.0, 1.0], t7["B"], t7["Q"], t7["e"])["
        C_actual"]), "passed": bool(constraint_metrics(t7)["feasible"])}
    cases["T8"] = {"cost": float(t8["C_actual"]), "passed": abs(t8["
        C_actual"] - 114.3) < 1e-6}
    return {"all_passed": all(v["passed"] for v in cases.values()), "cases"
        : cases}

if __name__ == "__main__":
    result = run_microcases()
    print(result)
    raise SystemExit(0 if result["all_passed"] else 1)
```

constraint_audit.py（全局约束审计脚本）

```
from __future__ import annotations
import json
import math
import argparse
from pathlib import Path
from params import FIGURE_DIR

def _walk(value, path="root"):
    if isinstance(value, dict):
        for key, item in value.items(): yield from _walk(item, f"{path}.{
            key}")
    elif isinstance(value, list):
        for index, item in enumerate(value): yield from _walk(item, f"{path
            }[{index}]")
```

```
    elif isinstance(value, (int, float)) and not isinstance(value, bool):
        yield path, value

def audit_file(path: Path) -> list[str]:
    errors = []
    data = json.loads(path.read_text(encoding="utf-8"))
    for location, value in _walk(data):
        if not math.isfinite(float(value)):
            errors.append(f"{path}:{location} 非有限数")
    if isinstance(data, dict) and "constraints_ok" in data and not data["
        constraints_ok"]:
        errors.append(f"{path}: constraints_ok=false")
    if isinstance(data, dict) and "daily" in data:
        for row in data["daily"]:
            if not row.get("audit", {}).get("feasible", False): errors.
                append(f"{path}:{row.get('date')} 物理约束失败")
            if not row.get("event_audit", False): errors.append(f"{path}:{
                row.get('date')} 事件来源分解失败")
    return errors

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("run_dir", nargs="?", default=None)
    args = parser.parse_args()
    root = Path(args.run_dir) if args.run_dir else FIGURE_DIR
    files = sorted(root.glob("*.json"))
    files = [p for p in files if p.name != "results.json"]
    errors = [error for path in files for error in audit_file(path)]
    if errors:
        for error in errors: print("ERROR", error)
        raise SystemExit(1)
    print(f"constraint audit PASS: {len(files)} JSON files")

if __name__ == "__main__":
    main()
```

scenario.py（情景树生成）

```
"""模型契约的最小可执行自检。运行本文件不会读取全年数据。"""
from __future__ import annotations
import numpy as np
from params import *

def build_tree(load_history, pv_history, price_history, max_leaves=16,
```

```
    block=36):
    """按历史联合净负载排序递归二分，生成确定性经验树。"""
    load_history = np.asarray(load_history, float); pv_history = np.asarray
        (pv_history, float); price_history = np.asarray(price_history, float
        )
    n = len(load_history)
    if n == 0:
        return [{"probability": 1.0, "load": None, "pv": None, "price":
            None, "sample_ids": []}]
    nodes = [np.arange(n)]; leaves = []
    while nodes:
        ids = nodes.pop(0)
        if len(ids) < 2 or len(leaves) + len(nodes) + 2 > max_leaves:
            leaves.append(ids); continue
        score = np.mean(load_history[ids] - pv_history[ids], axis=1)
        order = ids[np.argsort(score, kind="mergesort")]; midpoint = len(
            order) // 2
        nodes.extend([order[:midpoint], order[midpoint:]])
    result = []
    for ids in leaves:
        result.append({"probability": float(len(ids) / n), "load": np.mean(
            load_history[ids], axis=0).tolist(), "pv": np.mean(pv_history[
            ids], axis=0).tolist(), "price": np.mean(price_history[ids],
            axis=0).tolist(), "sample_ids": ids.tolist()})
    total = sum(row["probability"] for row in result)
    for row in result:
        row["probability"] /= total
    return result
```

sensitivity_analysis.py（参数灵敏度脚本）

```
from __future__ import annotations
import numpy as np
from data_model import load_data
from optimizer import solve_day_ahead
from params import FIGURE_DIR, POWER_MAX_KW, SEED, E_INITIAL_KWH
from utils import write_json

def run():
    data = load_data(); index = data.dates.index("2025-02-01")
    load, _ = data.history(index, "load"); pv, _ = data.history(index, "pv"
        ); price, _ = data.history(index, "price")
    values = [0.8 * POWER_MAX_KW, POWER_MAX_KW, 1.2 * POWER_MAX_KW]
    objectives = []
```

```
    for limit in values:
        import optimizer as _opt
        original = _opt.SLOT_POWER_LIMIT_KWH
        _opt.SLOT_POWER_LIMIT_KWH = limit / 6.0
        try:
            result = solve_day_ahead(load, pv, price, E_INITIAL_KWH)
            objectives.append(float(np.dot(result["p"], result["B"])))
        finally:
            _opt.SLOT_POWER_LIMIT_KWH = original
    output = {"seed": SEED, "parameter": "最大充放电功率(kW)", "values":
        values, "objective": objectives, "method": "same LP rerun;
        diagnostic only"}
    write_json(FIGURE_DIR / "sensitivity_results.json", output)
    return output

if __name__ == "__main__":
    run()
```

独立入口脚本：problem1.py / problem2.py / problem3.py / problem4.py / problem5.py / problem6.py

```
%% problem1.py
from __future__ import annotations
from data_model import load_data
from engine import run_q1
from params import *
from utils import write_json

def validate_constraints(result: dict) -> bool:
    return bool(result["constraints_ok"] and result["audit"]["feasible"]
        and result["audit"]["cycle"])

def run():
    result = run_q1(load_data())
    if not validate_constraints(result):
        raise ValueError("问题1约束审计失败")
    write_json(FIGURE_DIR / "problem_1_results.json", result)
    return result

if __name__ == "__main__":
    print("开始运行问题1")
    run()
    print("问题1完成")

```

```
%% problem2.py
from __future__ import annotations
from data_model import load_data
from engine import run_problem
from params import *
from utils import write_json

def validate_constraints(result: dict) -> bool:
    return bool(result["constraints_ok"])

def run():
    result = run_problem(load_data(), "q2")
    if not validate_constraints(result):
        raise ValueError("问题2约束审计失败")
    write_json(FIGURE_DIR / "problem_2_results.json", result)
    return result

if __name__ == "__main__":
    print("开始运行问题2")
    run()
    print("问题2完成")

%% problem3.py
from __future__ import annotations
from data_model import load_data
from engine import run_problem
from params import *
from utils import write_json

def validate_constraints(result: dict) -> bool:
    return bool(result["constraints_ok"])

def run():
    result = run_problem(load_data(), "q3")
    if not validate_constraints(result):
        raise ValueError("问题3约束审计失败")
    write_json(FIGURE_DIR /
\subsection*{绘图脚本（PDF图表生成源码）}
\begin{lstlisting}
%% gen_fig_q1_dispatch.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
```

```
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import q1, time_hours
d = q1(); x = time_hours()
kw = {k: np.asarray(d[k], dtype=float) * 6.0 for k in ['load_kwh','pv_kwh',
    'plan','charge','discharge']}
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.fill_between(x, 0, kw['load_kwh'], color=PALETTE[0], alpha=0.18, label='
    负载功率')
ax.plot(x, kw['load_kwh'], color=PALETTE[0], linewidth=1.5)
ax.fill_between(x, 0, kw['pv_kwh'], color=PALETTE[2], alpha=0.28, label='光
    伏功率')
ax.plot(x, kw['pv_kwh'], color=PALETTE[2], linewidth=1.4)
ax.plot(x, kw['plan'], color=PALETTE[1], linewidth=1.6, label='计划购电')
ax.plot(x, kw['discharge'], color=PALETTE[3], linewidth=1.2, label='储能放
    电')
ax.plot(x, kw['charge'], color=PALETTE[5], linewidth=1.2, linestyle='--',
    label='储能充电')
ax.set_xlabel('日内时间␣(h)'); ax.set_ylabel('功率␣(kW)')
ax.grid(axis='y', alpha=.12, linestyle='--', color=COLORS['grid'])
ax.set_xticks(np.arange(0, 25, 4)); ax.set_xlim(0, 24)
ax.legend(ncol=5, fontsize=8, loc='upper␣center', bbox_to_anchor=(.5, .99),
          frameon=False, borderaxespad=0, columnspacing=1.2, handlelength
              =2.0)
ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
fig.subplots_adjust(left=.10, right=.97, bottom=.14, top=.90); save_fig(fig
    , 'figures/fig_q1_dispatch.pdf')
```

使用 Python 生成的图片源码如下:

```
%% gen_fig_q1_soc_price.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
```

```
setup_style()
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import q1, time_hours
d = q1(); x = time_hours(endpoint=True)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6.5), sharex=True)
ax1.fill_between(x, 1200, np.asarray(d['soc']), color=PALETTE[0], alpha
    =.08)
ax1.grid(axis='y', alpha=.12, color=COLORS['grid'])
ax1.plot(x, d['soc'], color=PALETTE[0], linewidth=2.0, label='储能电量')
ax1.axhline(1200, color=COLORS['ref_line'], linestyle=':', linewidth=1,
    label='运行下限')
ax1.axhline(10800, color=COLORS['ref_line'], linestyle='--', linewidth=1,
    label='运行上限')
ax1.set_ylabel('储能电量␣(kWh)'); ax1.legend(fontsize=8, ncol=3, loc='lower
    ␣center',
     bbox_to_anchor=(.5, 1.01), frameon=False, borderaxespad=0,
         columnspacing=1.4)
ax2.plot(time_hours(), d['price'], color=PALETTE[1], linewidth=1.7, label='
    固定电价')
charge = np.asarray(d['charge']) > 1e-6; discharge = np.asarray(d['
    discharge']) > 1e-6
ax2.scatter(time_hours()[charge], np.asarray(d['price'])[charge], s=13,
    color=PALETTE[2], edgecolors='white', label='充电时段', zorder=3)
ax2.scatter(time_hours()[discharge], np.asarray(d['price'])[discharge], s
    =13, color=PALETTE[4], edgecolors='white', label='放电时段', zorder=3)
ax2.set_xlabel('日内时间␣(h)'); ax2.set_ylabel('电价␣(元/kWh)'); ax2.legend
    (fontsize=8, ncol=3, loc='lower␣center',
     bbox_to_anchor=(.5, 1.01), frameon=False, borderaxespad=0,
         columnspacing=1.4)
ax1.grid(axis='y', alpha=.12, color=COLORS['grid'])
ax2.grid(axis='y', alpha=.12, color=COLORS['grid'])
ax1.spines['top'].set_visible(False); ax1.spines['right'].set_visible(False
    )
ax2.spines['top'].set_visible(False); ax2.spines['right'].set_visible(False
    )
ax2.set_xticks(np.arange(0, 25, 4)); fig.subplots_adjust(left=.11, right
    =.97, bottom=.12, top=.94, hspace=.28); save_fig(fig, 'figures/
    fig_q1_soc_price.pdf')
```

```
%% gen_fig_q2_monthly_cost.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
```

```
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import month_rows
rows = month_rows('q2'); months = [r['month'] for r in rows]; cost = [r['cost'] for r in rows]; emergency = [r['emergency_kwh'] for r in rows]
fig, ax1 = plt.subplots(figsize=(8.5, 4.5)); x = np.arange(len(rows))
bars = ax1.bar(x, cost, color=PALETTE[0], alpha=.62, edgecolor='white', linewidth=1.1, label='实际费用')
ax1.fill_between(x, 0, cost, color=PALETTE[0], alpha=.08)
month_labels = [f'{int(month[-2:])}月' for month in months]
ax1.set_ylabel('月实际费用␣(元)', color=PALETTE[0]); ax1.set_xlabel(f'月份（{len(rows)}␣个月，全量交付期）'); ax1.set_xticks(x); ax1.set_xticklabels(month_labels, fontsize=9)
ax2 = ax1.twinx(); ax2.plot(x, emergency, 'o-', color=PALETTE[1], linewidth=2, markersize=6, markeredgecolor='white', label='紧急购电量')
ax2.set_ylabel('紧急购电量␣(kWh)', color=PALETTE[1]); ax1.tick_params(axis='y', labelcolor=PALETTE[0]); ax2.tick_params(axis='y', labelcolor=PALETTE[1])
for bar, value in zip(bars, cost): ax1.text(bar.get_x()+bar.get_width()/2, value, f'{value/1000:.1f}k', ha='center', va='bottom', fontsize=8, bbox=dict(boxstyle='round,pad=.12', facecolor='white', edgecolor='none', alpha=.75))
lines1, labs1 = ax1.get_legend_handles_labels(); lines2, labs2 = ax2.get_legend_handles_labels(); ax1.legend(lines1+lines2, labs1+labs2, fontsize=8, loc='upper␣left')
ax2.spines['right'].set_visible(False); ax1.spines['top'].set_visible(False); ax2.spines['top'].set_visible(False); ax1.grid(axis='y', alpha=.12, color=COLORS['grid'])
fig.tight_layout(); save_fig(fig, 'figures/fig_q2_monthly_cost.pdf')
```

```
%% gen_fig_q2_soc_sample.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p):
```

```
            shutil.copy2(p, f'_utils/{src}')
            break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import daily_rows, time_hours
rows = daily_rows('q2')
target_dates = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
selected = {day['date']: day for day in rows if day['date'] in target_dates}
missing = [date for date in target_dates if date not in selected]
if missing:
    raise ValueError(f'指定日期缺失: {missing}')
fig, axes = plt.subplots(2, 2, figsize=(10, 7), sharex=True)
x_state = time_hours(endpoint=True)
x_interval = time_hours()
for ax, date in zip(axes.flat, target_dates):
    day = selected[date]
    tr = day['trajectory']
    soc = np.asarray(tr['E'], dtype=float)
    net_load = (np.asarray(tr['L'], dtype=float) - np.asarray(tr['V'],
        dtype=float)) * 6.0
    ax2 = ax.twinx()
    ax.plot(x_state, soc, color=PALETTE[0], linewidth=1.7, label='储能电量'
        )
    ax2.plot(x_interval, net_load, color=PALETTE[1], linewidth=1.2, label='
        净负载')
    ax.axhline(1200, color=COLORS['ref_line'], linestyle=':', linewidth=.9)
    ax.axhline(10800, color=COLORS['ref_line'], linestyle=':', linewidth
        =.9)
    ax.text(.03, .94, date, transform=ax.transAxes, fontsize=9, va='top',
            bbox=dict(boxstyle='round,pad=.2', facecolor='white',
                      edgecolor=COLORS['grid'], alpha=.85))
    ax.set_ylabel('SOC (kWh)')
    ax2.set_ylabel('净负载 (kW)', color=PALETTE[1])
    ax.set_xlim(0, 24)
    ax.set_xticks(np.arange(0, 25, 4))
    ax.grid(axis='y', alpha=.10, color=COLORS['grid'])
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax2.spines['top'].set_visible(False)
```

```
    ax2.tick_params(axis='y', labelcolor=PALETTE[1])
for ax in axes[1, :]:
    ax.set_xlabel('日内时间 (h)')
handles, labels = axes.flat[0].get_legend_handles_labels()
handles2 = [Line2D([0], [0], color=PALETTE[1], linewidth=1.2)]
labels2 = ['净负载']
fig.legend(handles + handles2, labels + labels2, loc='upper center', ncol=1,
           bbox_to_anchor=(.5, .995), frameon=False, fontsize=9,
           labelspacing=.35, handlelength=2.0)
fig.tight_layout(rect=(0, 0, 1, .88))
save_fig(fig, 'figures/fig_q2_soc_sample.pdf')
```

```
%% gen_fig_q3_forecast_error.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import forecast_metrics
fc = forecast_metrics(); first_date = sorted(fc['date'].unique())[0]
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.5))
for issue, color in zip([0, 6, 12, 18], PALETTE[:4]):
    sub = fc[(fc['date'] == first_date) & (fc['issue_hour'] == issue) & (fc['lead'] <= 12)].sort_values('lead')
    if len(sub): ax1.plot(sub['lead'], sub['forecast'], color=color, linewidth=1.6, marker='o', markersize=3, markeredgecolor='white', label=f'{issue:02d}:00 预测')
sub = fc[(fc['date'] == first_date) & (fc['issue_hour'] == 0) & (fc['lead'] <= 12)].sort_values('lead')
if len(sub): ax1.plot(sub['lead'], sub['actual'], color=COLORS['text'], linestyle='--', linewidth=1.5, marker='o', markersize=3, markeredgecolor='white', label='实际值')
ax1.grid(axis='y', alpha=.12, color=COLORS['grid'])
ax1.set_xlabel('提前量 (h)'); ax1.set_ylabel('光伏功率 (kW)'); ax1.text(.02,.96,first_date,transform=ax1.transAxes,fontsize=9,va='top',color=COLORS['text'])
```

```
issue_mae = fc.groupby('issue_hour')['abs_error'].mean(); ax2.bar(issue_mae.index.astype(str), issue_mae.values, color=PALETTE[1], alpha=.65, edgecolor=PALETTE[1]); ax2.set_xlabel('发行时刻'); ax2.set_ylabel('平均绝对误差␣(kW)')
ax2.text(.98, .95, f'有效配对␣{len(fc):,}␣条', transform=ax2.transAxes, ha='right', va='top', fontsize=8, color=COLORS['text'])
for ax in (ax1, ax2): ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
handles, labels = ax1.get_legend_handles_labels()
for xpos, text, color in zip((.16, .28, .40), labels[:3], PALETTE[:3]):
    fig.text(xpos, .975, text, ha='center', va='top', fontsize=8, color=color)
for xpos, text, color in zip((.23, .35), labels[3:], (PALETTE[3], COLORS['text'])):
    fig.text(xpos, .935, text, ha='center', va='top', fontsize=8, color=color)
fig.subplots_adjust(left=.09, right=.97, bottom=.16, top=.83, wspace=.28)
save_fig(fig, 'figures/fig_q3_forecast_error.pdf')
```

```
%% gen_fig_q4_price_sensitivity.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mc
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import load_results
base=load_results()['q4_q3']; total=float(base['C_actual']); up=float(base['C_up']); down=float(base['C_down']); emergency=float(base['C_emergency']); labels=['整体价格␣+10%','整体价格␣-10%','紧急倍率␣+1','紧急倍率␣-1','上调倍率␣+10%','下调费率␣+10%']; low=np.array([-0.10*total,0.10*total,-emergency/5,emergency/5,-0.10*up,-0.10*down]); high=np.array([0.10*total,-0.10*total,emergency/5,-emergency/5,0.10*up,0.10*down]); impact=np.column_stack([low,high]); order=np.argsort(np.max(np.abs(impact),axis=1)); labels=[labels[i] for i in order]; low=low[order]; high=high[order]; y=np.arange(len(labels)); fig,ax=plt.subplots(figsize=(8,4.5)); ax.barh(y, np.maximum(low,0), color=PALETTE[1], alpha=.58, edgecolor=PALETTE[1],
```

```
    label='正向费用变化'); ax.barh(y, np.minimum(high,0), color=PALETTE[0],
    alpha=.58, edgecolor=PALETTE[0]);
for yi,lo,hi in zip(y,low,high):
    ax.plot([min(lo,hi),max(lo,hi)],[yi,yi],color=COLORS['ref_line'],
        linewidth=.8); ax.text(hi + (abs(hi)*.04+1), yi, f'{hi:+,.0f}', va='
        center', fontsize=8)
ax.axvline(0,color=COLORS['text'],linewidth=1); ax.grid(axis='x',alpha=.12,
    color=COLORS['grid']); ax.set_yticks(y); ax.set_yticklabels(labels); ax.
    set_xlabel('相对基准实付费用变化␣(元)'); ax.spines['top'].set_visible(
    False); ax.spines['right'].set_visible(False)
fig.text(.96, .155, '正值表示费用增加', ha='right', va='bottom', fontsize
    =8,
         color=COLORS['text'])
fig.subplots_adjust(left=.18, right=.96, bottom=.16, top=.90)
save_fig(fig,'figures/fig_q4_price_sensitivity.pdf')
```

```
%% gen_fig_q4_scenario_compare.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patheffects as pe
from matplotlib.colors import Normalize
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import daily_cost_rows, load_results
spec = [('q2','固定价␣q2'),('q3_dispatch','固定价␣q3'),('q4_q2','波动价␣q4-
    q2'),('q4_q3','波动价␣q4-q3')]; metric_names = ['实际费用','紧急电量','
    充电量','日末␣SOC']; matrix=[]
for group,_ in spec:
    rows=daily_cost_rows(group); matrix.append([np.mean([r['actual'] for r
        in rows]), np.mean([r['emergency_kwh'] for r in rows]), np.mean([r['
        charge_kwh'] for r in rows]), np.mean([r['soc_end'] for r in rows])
        ])
arr=np.asarray(matrix); normed=(arr-arr.min(axis=0))/(np.ptp(arr,axis=0)+1e
    -12); fig, ax=plt.subplots(figsize=(8.6,4.8)); im=ax.imshow(normed, cmap
    ='YlOrBr', vmin=0, vmax=1, aspect='auto'); ax.set_xticks(range(4)); ax.
    set_xticklabels(metric_names, fontsize=9.5); ax.set_yticks(range(4)); ax
```

```
    .set_yticklabels(['固定价␣q2', '固定价␣q3', '波动价\nq4-q2', '波动价\nq4
    -q3'], fontsize=9.5, linespacing=1.05); ax.tick_params(axis='y', pad=5)
for i in range(4):
    for j in range(4):
        cell_color = im.cmap(im.norm(normed[i, j]))
        luminance = 0.2126 * cell_color[0] + 0.7152 * cell_color[1] +
            0.0722 * cell_color[2]
        text_color = 'white' if luminance < 0.56 else COLORS['text']
        outline_color = COLORS['text'] if text_color == 'white' else 'white
            '
        ax.text(j, i, f'{arr[i,j]:,.0f}', ha='center', va='center',
            fontsize=8.5,
                color=text_color,
                path_effects=[pe.withStroke(linewidth=1.4, foreground=
                    outline_color)])
cbar=fig.colorbar(im,ax=ax,pad=.03); cbar.ax.tick_params(labelsize=8); ax.
    grid(color=COLORS['grid'], alpha=.10); fig.subplots_adjust(left=.24,
    right=.90, bottom=.15, top=.93); save_fig(fig,'figures/
    fig_q4_scenario_compare.pdf')
```

```
%% gen_fig_model_validation.py
import os, sys, shutil
os.makedirs('_utils', exist_ok=True)
for src in ['plot_utils.py']:
    for search in ['skills/shared-scripts', '../skills/shared-scripts']:
        p = os.path.join(search, src)
        if os.path.isfile(p): shutil.copy2(p, f'_utils/{src}')
        break
sys.path.insert(0, '.')
from _utils.plot_utils import setup_style, save_fig, PALETTE, COLORS
setup_style()
import json
import numpy as np
import matplotlib.pyplot as plt
sys.path.insert(0, os.path.dirname(__file__))
from _figure_common import load_results, daily_rows
root = load_results(); labels = ['问题1','问题2','问题3','问题4-q2','问题4-
    q3']; groups = ['q1','q2','q3_dispatch','q4_q2','q4_q3']; residual=[];
    gaps=[]; costs=[]
for group in groups:
    d=root[group]
    costs.append(float(d.get('billing',{}).get('C_actual', d.get('C_actual'
        , d.get('total_cost',0)))))
    if group=='q1':
        residual.append(max(float(d['audit']['max_balance_residual']),
```

```
            float(d['audit']['max_soc_residual']))); gaps.append(0.0)
    else:
        rows=daily_rows(group); residual.append(max(float(x.get('audit',{})
            .get('max_balance_residual',0)) for x in rows)); gaps.append(max
            (float(x.get('solver',{}).get('mip_gap',0) or 0) for x in rows))
fig, ax1 = plt.subplots(figsize=(6.8,4.5)); x=np.arange(len(labels)); ax1.
    bar(x,residual,color=PALETTE[0],alpha=.68,edgecolor=PALETTE[0]); ax1.
    grid(axis='y', alpha=.12, color=COLORS['grid']); ax1.axhspan(0, max(
    residual) if max(residual) > 0 else 1, color=PALETTE[0], alpha=.04); ax1
    .set_yscale('symlog',linthresh=1e-14); ax1.set_xticks(x); ax1.
    set_xticklabels(labels,rotation=25,ha='right'); ax1.set_ylabel('最大约束
    残差␣(kWh)'); ax1.set_xlabel('运行情形'); ax1.axhline(1e-6,color=COLORS[
    'ref_line'],linestyle='--',linewidth=1,label='审计容差'); ax1.legend(
    fontsize=8,loc='upper␣right',bbox_to_anchor=(.98,.90),borderaxespad=0)
ax1.spines['top'].set_visible(False); ax1.spines['right'].set_visible(False
    )
fig.subplots_adjust(left=.15, right=.96, bottom=.22, top=.94); save_fig(fig
    ,'figures/fig_model_validation.pdf')
```